What is JavaScript

JavaScript is a programming language that executes on the browser. It turns static HTML web pages into interactive web pages by dynamically updating content, validating form data, controlling multimedia, animating images, and almost everything else on the web pages

JavaScript is the third most important web technology after HTML and CSS. JavaScript can be used to create web and mobile applications, build web servers, create games, etc.

JavaScript History

In early 1995, Brendan Eich from Netscape designed and implemented a new language for non-java programmers to give newly added Java support in Netscape navigator. It was initially named Mocha, then LiveScript, and finally JavaScript.

Nowadays, JavaScript can execute not only on browsers but also on the server or any device with a JavaScript Engine. For example, Node.js is a framework based on JavaScript that executes on the server.

JavaScript and ECMAScript

Often you will hear the term ECMAScript while working with JavaScript.

As you now know, JavaScript was primarily developed to execute on browsers. There are many different browsers from different companies. So, there was a need to standardize the execution of the JavaScript code to achieve the same functionality in all the browsers.

Ecma International is a non-profit organization that creates standards for technologies. ECMA International publishes the specification for scripting languages is called 'ECMAScript'. ECMAScript specification defined in ECMA-262 for creating a general-purpose scripting language.

JavaScript implements the ECMAScript standards, which includes features specified in ECMA-262 specification as well as other features which are not based on ECMAScript standards.

There are different editions of ECMAScript. Most browsers have implemented ECMA-262 5.1 edition.

- ECMA-262 5.1 edition, June 2011
- ECMA-262, 6th edition, June 2015
- ECMA-262, 7th edition, June 2016
- ECMA-262, 8th edition, June 2017
- ECMA-262, 9th edition, June 2018
- ECMA-262, 10th edition, June 2019
- ECMA-262, 11th edition, June 2020
- ECMA-262, 12th edition, June 2021
- ECMA-262, 13th edition, June 2022
- ECMA-262, 14th edition, June 2023
- ECMA-262, 15th edition, June 2024
- ECMA-262, 16th edition, June 2025

JavaScript Engine

JavaScript engine interprets, compiles, and executes JavaScript code. It also does memory management, JIT compilation, type system etc. Different browsers use different JavaScript engines, as listed in the below table.

Browser	JavaScript Engine
Edge	chakra
Chrome	V8
FireFox	SpiderMonkey
Safari	Nitro

Setup JavaScript Development Environment

Here, we are going to use JavaScript in developing a web application. So, we must have at least two things, a browser, and an editor to write the JavaScript code.

Although we also need a webserver to run a web application, but we will use a single HTML web page to run our JavaScript code. So, no need to install it for now.

Browser

Mostly, you will have a browser already installed on your PC, Microsoft Edge on the Windows platform, and Safari on Mac OS.

You can also install the following browser as per your preference:

Microsoft Edge
Google Chrome
Mozilla FireFox
Safari
Opera

IDEs for JavaScript Application Development

You can write JavaScript code using a simple editor like Notepad. However, you can install any open-sourced or licensed IDE (Integrated Development Environment) to get the advantage of IntelliSense support for JavaScript and syntax error/warning highlighter for rapid development.

The followings are some of the well-known JavaScript editors:

Visual Studio Code (Free, cross-platform)
Eclipse (Free, cross-platform)
Atom (Free, cross-platform)
Notepad++ (Free, Windows)
Code Lobster (Free, cross-platform)
WebStorm (Paid, cross-platform)

Online Editors for JavaScript

Use the online editor to quickly execute the JavaScript code without any installation. The followings are free online editors:

- jsfiddle.net
- jsbin.com
- playcode.io

HTML script Tag

The HTML script tag <script> is used to embed data or executable client side scripting language in an HTML page. Mostly, JavaScript or JavaScript based API code inside a `<script></script>` tag.

The following is an example of an HTML page that contains the JavaScript code in a `<script>` tag.

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
    <h1> JavaScript Tutorials</h1>

    <script>
        //write JavaScript code here..
        alert('Hello, how are you?')
    </script>
</body>
</html>
```

In the above example, a `<script></script>` tag contains the JavaScript `alert('Hello, how are you?')` that display a message box.

HTML v4 requires the `type` attribute to identify the language of script code embedded within script tag. This is specified as MIME type e.g. 'text/javascript', 'text/ecmascript', 'text/vbscript', etc.

HTML v5 page does not require the `type` attribute because the default script language is 'text/javascript' in a `<script>` tag.

An HTML page can contain multiple `<script>` tags in the `<head>` or `<body>` tag. The browser executes all the script tags, starting from the first script tag from the beginning.

Scripts without `async`, `defer` or `type="module"` attributes, as well as inline scripts, are fetched and executed immediately, before the browser continues to parse the page. Consider the following page with multiple script tags.

```
<!DOCTYPE html>
<html>
<head>
    <script>
        alert('Executing JavaScript 1')
    </script>
</head>
<body>
    <h1> JavaScript Tutorials</h1>

    <script>
        alert('Executing JavaScript 2')
    </script>

    <p>This page contains multiple script tags.</p>

    <script>
        alert('Executing JavaScript 3')
    </script>
</body>
</html>
```

Above, the first `<script>` tag containing `alert('Executing JavaScript 1')` will be executed first, then `alert('Executing JavaScript 2')` will be executed, and then `alert('Executing JavaScript 3')` will be executed.

The browser loads all the scripts included in the `<head>` tag before loading and rendering the `<body>` tag elements. So, always include JavaScript files/code in the `<head>` that are going to be used while rendering the UI. All other scripts should be placed before the ending `</body>` tag. This way, you can increase the page loading speed.

Reference the External Script File

A `<script>` tag can also be used to include an external script file to an HTML web page by using the `src` attribute.

If you don't want to write inline JavaScript code in the `<script></script>` tag, then you can also write JavaScript code in a separate file with `.js` extension and include it in a web page using <script> tag and reference the file via src attribute.

```
<!DOCTYPE html>
<html>
<head>
    <script src="/MyJavaScriptFile.js" ></script>
</head>
<body>
    <h1> JavaScript Tutorials</h1>

</body>
</html>
```

Above, the `<script src="/MyJavaScriptFile.js">` points to the external JavaScript file using the `src="/MyJavaScriptFile.js"` attribute

where the value of the `src` attribute is the path or url from which a file needs to be loaded in the browser. Note that you can load the files from your domain as well as other domains.

Global Attributes

The `<script>` can contain the following global attributes:

Attribute	Usage
async	`<script async>` executes the script asynchronously along with the rest of the page.
crossorign	`<script crossorigin="anonymous|use-credentials">` allows error logging for sites which use a separate domain for static media. Value `anonymous` do not send credentials, whereas `use-credentials` sends the credentials.
defer	`<script defer>` executes the script after the document is parsed and before firing DOMContentLoaded event.
src	`<script src="uri\path to resource">` specifies the URI/path of an external script file;
type	`<script type="text\javascript">` specifies the type of the containing script e.g. text\javascript, text\html, text\plain, application\json, application\pdf, etc.

referrerpolicy	`<script referrerpolicy="no-referrer">` specifies which referrer information to send when fetching a script. Values can be no-referrer, no-referrer-when-downgrade, origin, same-origin, strict-origin, etc.
integrity	`<script integrity="sha384-oqVuAfXRKap7fdgc">` specifies that a user agent can use to verify that a fetched resource has been delivered free of unexpected manipulation.
nomodule	`<script nomodule>` specifies that the script should not be executed in browsers supporting ES2015 modules.

JavaScript Syntax

```
<script> //Write javascript code here...

</script>
```

Character Set

JavaScript uses the unicode character set, so allows almost all characters, punctuations, and symbols.

Case Sensitive

JavaScript is a case-sensitive scripting language. So, name of functions, variables and keywords are case sensitive. For example, `myfunction` and `MyFunction` are different, `Name` is not equal to `nAme`, etc.

Variables

In JavaScript, a variable is declared with or without the `var` keyword.

```
<script>
var name = "Steve";
id = 10;
</script>
```

Whitespaces

JavaScript ignores multiple spaces and tabs. The following statements are the same.

```
<script>
     var one =1;
     var one = 1;
     var one = 1;
</script>
```

Code Comments

A comment is single or multiple lines, which give some information about the current program. Comments are not for execution.

Write comment after double slashes `//` or write multiple lines of comments between `/*` and `*/`

```
<script>
     var one =1;
     // this is a single line comment
     /* this is multi line comment*/
     var two = 2;
     var three = 3;
```

```
</script>
```

String

A string is a text in JavaScript. The text content must be enclosed in double or single quotation marks.

```
<script>
var msg = "Hello World" //JavaScript string in double quotes
var msg = 'Hello World' //JavaScript string in single quotes
</script>
```

Number

JavaScript allows you to work with any type of number like integer, float, hexadecimal etc. Number must NOT be wrapped in quotation marks.

```
<script>
     var num = 100;
     var flot = 10.5;
</script>
```

Boolean

As in other languages, JavaScript also includes `true` and `false` as a boolean value.

```
<script>
     var yes = true;
     var no = false;
```

```
</script>
```

Keywords

Keywords are reserved words in JavaScript, which cannot be used as variable names or function names.

The following table lists some of the keywords used in JavaScript.

JavaScript Reserved Keywords		
var	function	if
else	do	while
for	switch	break
continue	return	try
catch	finally	debugger
case	class	this
default	false	true
in	instanceOf	typeOf
new	null	throw
void	width	delete

JavaScript Message Boxes: alert(), prompt(), confirm(),

JavaScript provides built-in global functions to display popup message boxes for different purposes.

> alert(message): Display a popup box with the specified message with the OK button.
> confirm(message): Display a popup box with the specified message with OK and Cancel buttons.
> prompt(message, defaultValue): Display a popup box to take the user's input with the OK and Cancel buttons.

alert()

The `alert()` function displays a message to the user to display some information to users. This alert box will have the OK button to close the alert box.

The `alert()` function takes a parameter of any type e.g., string, number, boolean etc. So, no need to convert a non-string type to a string type.

```
alert("This is an alert message box.");
// display string message
alert('This is a number: ' + 100);
// display result of a concatenation
alert(100); // display number
alert(Date()); // display current date
```

confirm()

Use the `confirm()` function to take the user's confirmation before starting some task. For example, you want to take the user's confirmation before saving, updating or deleting data.

The `confirm()` function displays a popup message to the user with two buttons, `OK` and `Cancel`. The `confirm()` function returns `true` if a user has clicked on the `OK` button or returns `false` if clicked on the `Cancel` button. You can use th

e return value to process further.

The following takes user's confirmation before saving data:

```
var userPreference;
if (confirm("Do you want to save changes?") == true)
{
     userPreference = "Data saved successfully!";
}
else
{
     userPreference = "Save Cancelled!";
}
```

prompt()

Use the `prompt()` function to take the user's input to do further actions. For example, use the `prompt()` function in the scenario where you want to calculate EMI based on the user's preferred loan tenure.

The `prompt()` function takes two parameters. The first parameter is the message to be displayed, and the second parameter is the default value in an input box.

```
var name = prompt("Enter your name:", "John");
if (name == null || name == "")
{
     document.getElementById("msg").innerHTML = "You did not enter anything. Please enter your name again";
     }
     else
      {
     document.getElementById("msg").innerHTML = "You entered: " + name;
     }
```

JavaScript Variables

Variable means anything that can vary. In JavaScript, a variable stores data that can be changed later on.

Declare a Variable

In JavaScript, a variable can be declared using `var`, `let`, `const` keywords.

> var keyword is used to declare variables since JavaScript was created. It is confusing and error-prone when using variables declared using `var`.
> let keyword removes the confusion and error of `var`. It is the new and recommended way of declaring variables in JavaScript.
> const keyword is used to declare a constant variable that cannot be changed once assigned a value.

Here, we will use the `let` keyword to declare variables. To declare a variable, write the keyword `let` followed by the name of the variable you want to give, as shown below.

```
let msg; // declaring a variable without assigning a value
```

in the above example, `var msg;` is a variable declaration. It does not have any value yet. The default value of variables that do not have any value is undefined.

You can assign a value to a variable using the `=` operator when you declare it or after the declaration and before accessing it.

```
let msg;

msg = "Hello JavaScript!"; // assigning a string value
```

In the above example, the `msg` variable is declared first and then assigned a string value in the next line.

You can declare a variable and assign a value to it in the same line. Values can be of any datatype such as string, numeric, boolean, etc.

```
let name = "Steve"; //assigned string value

let num = 100; //assigned numeric value

let isActive = true; //assigned boolean value
```

Multiple variables can be declared in a single line, as shown below.

```
let name = "Steve", num = 100, isActive = true;
```

You can copy the value of one variable to another variable, as shown below.

```
let num1 = 100;

let num2 = num1;
```

JavaScript allows multiple white spaces and line breaks when you declare a variables.

```
let name = "Steve",

    num = 100,

    isActive = true;
```

Variable names are case-sensitive in JavaScript. You cannot declare a duplicate variable using the `let` keyword with the same name and case. JavaScript will throw a syntax error. Although, variables can have the same name if declared with the `var` keyword (this is why it is recommended to use `let`).

```
let num = 100;

let num = 200; //syntax error



var num = 100;

var num = 200; //Ok
```

JavaScript Variable Naming Conventions

Variable names are case-sensitive in JavaScript. So, the variable names `msg`, `MSG`, `Msg`, `mSg` are considered separate variables.
Variable names can contain letters, digits, or the symbols $ and _.
A variable name cannot start with a digit 0-9.
A variable name cannot be a reserved keyword in JavaScript, e.g. var, function, return cannot be variable names.
Space not allowed.

Dynamic Typing

JavaScript is a loosely typed language. It means that you don't need to specify what data type a variable will contain. You can update the value of any type after initialization. It is also called dynamic typing.

```
let myVariable = 1;  // numeric value


myVariable = 'one'; // string value


myVariable = 1.1; // decimal value


myVariable = true; // Boolean value


myVariable = null; // null value
```

Constant Variables in JavaScript

Use `const` keyword to declare a constant variable in JavaScript.

> Constant variables must be declared and initialized at the same time.
> The value of the constant variables can't be changed after initialized them.

```
const num = 100;
```

```
num = 200; //error



const name; //error

name = "Steve";
```

The value of a constant variable cannot be changed but the content of the value can be changed. For example, if an object is assigned to a const variable then the underlying value of an object can be changed.

```
const person = { name: 'Steve'};

person.name = "Bill";

alert(person.name); //Bill
```

It is best practice to give constant variable names in capital letters to separate them from other non-constant variables.

Variable Scope

In JavaScript, a variable can be declared either in the global scope or the local scope.

Global Variables

Variables declared out of any function are called global variables. They can be accessed anywhere in the JavaScript code, even inside any function.

Local Variables

Variables declared inside the function are called local variables of that function. They can only be accessed in the function where they are declared but not outside.

The following example includes global and local variables.

```
let greet = "Hello " // global variable

function myfunction(){

    let msg = "JavaScript!";

    alert(greet + msg); //can access global and local variable

}



myfunction();



alert(greet);//can access global variable

alert(msg); //error: can't access local variable
```

Declare Variables without var and let Keywords

Variables can be declared and initialized without the `var` or `let` keywords. However, a value must be assigned to a variable declared without the `var` keyword.

The variables declared without the `var` keyword become global variables, irrespective of where they are declared.

It is Recommended to declare variable using the `let` keyword.

```
function myfunction(){

     msg = "Hello JavaScript!";

}



myfunction();

alert(msg); // msg becomes global variable so can be accessed here
```

1.Variables can be defined using let keyword. Variables defined without let or var keyword become global variables.
2.Variables should be initialized before accessing it. Unassigned variable has value undefined.
3.JavaScript is a loosely-typed language, so a variable can store any type value.
4.Variables can have local or global scope. Local variables cannot be accessed out of the function where they are declared, whereas the global variables can be accessed from anywhere.

Javascript Operators

JavaScript includes operators same as other languages. An operator performs some operation on single or multiple operands (data value) and produces a result. For example, in `1 + 2`, the `+` sign is an operator and 1 is left side operand and 2 is right side operand. The `+` operator performs the addition of two numeric values and returns a result.

JavaScript includes following categories of operators.

Arithmetic Operators

Arithmetic operators are used to perform mathematical operations between numeric operands.

Operator	Description
+	Adds two numeric operands.
-	Subtract right operand from left operand
*	Multiply two numeric operands.
/	Divide left operand by right operand.
%	Modulus operator. Returns remainder of two operands.
++	Increment operator. Increase operand value by one.
--	Decrement operator. Decrease value by one.

The following example demonstrates how arithmetic operators perform different tasks on operands.

```
let x = 5, y = 10;
let z = x + y; //performs addition and returns 15
z = y - x; //performs subtraction and returns 5
```

```
z = x * y; //performs multiplication and returns 50
z = y / x; //performs division and returns 2
z = x % 2; //returns division remainder 1
```

The `++` and `--` operators are unary operators. It works with either left or right operand only. When used with the left operand, e.g., `x++`, it will increase the value of `x` when the program control goes to the next statement. In the same way, when it is used with the right operand, e.g., `++x`, it will increase the value of `x` there only. Therefore, `x++` is called post-increment, and `++x` is called pre-increment.

```
let x = 5;
x++; //post-increment, x will be 5 here and 6 in the next line
++x; //pre-increment, x will be 7 here
x--; //post-decrement, x will be 7 here and 6 in the next line
--x; //pre-decrement, x will be 5 here
```

String Concatenation

The `+` operator performs concatenation operation when one of the operands is of string type. The following example demonstrates string concatenation even if one of the operands is a string.

```
let a = 5, b = "Hello ", c = "World!", d = 10;
a + b; //returns "5Hello "
b + c; //returns "Hello World!"
a + d; //returns 15
b + true; //returns "Hello true"
c - b; //returns NaN; - operator can only used with numbers
```

Comparison Operators

JavaScript provides comparison operators that compare two operands and return a boolean value `true` or `false`.

Operators	Description
==	Compares the equality of two operands without considering type.
===	Compares equality of two operands with type.
!=	Compares inequality of two operands.
>	Returns a boolean value true if the left-side value is greater than the right-side value; otherwise, returns false.
<	Returns a boolean value true if the left-side value is less than the right-side value; otherwise, returns false.
>=	Returns a boolean value true if the left-side value is greater than or equal to the right-side value; otherwise, returns false.
<=	Returns a boolean value true if the left-side value is less than or equal to the right-side value; otherwise, returns false.

The following example demonstrates the comparison operators.

```
let a = 5, b = 10, c = "5";
let x = a;
a == c; // returns true
a === c; // returns false
a == x; // returns true
a != b; // returns true
a > b; // returns false
a < b; // returns true
a >= b; // returns false
a <= b; // returns true
```

Logical Operators

In JavaScript, the logical operators are used to combine two or more conditions. JavaScript provides the following logical operators.

Operator	Description
&&	&& is known as AND operator. It checks whether two operands are non-zero or not (0, false, undefined, null or "" are considered as zero). It returns 1 if they are non-zero; otherwise, returns 0.
||	|| is known as OR operator. It checks whether any one of the two operands is non-zero or not (0, false, undefined, null or "" is considered as zero). It returns 1 if any one of of them is non-zero; otherwise, returns 0.
!	! is known as NOT operator. It reverses the boolean result of the operand (or condition). `!false` returns `true`, and `!true` returns `false`.

```
let a = 5, b = 10;
(a != b) && (a < b); // returns true
(a > b) || (a == b); // returns false
(a < b) || (a == b); // returns true
!(a < b); // returns false
!(a > b); // returns true
```

Assignment Operators

JavaScript provides the assignment operators to assign values to variables with less key strokes.

Assignment operators	Description
=	Assigns right operand value to the left operand.

+=	Sums up left and right operand values and assigns the result to the left operand.
-=	Subtract right operand value from the left operand value and assigns the result to the left operand.
*=	Multiply left and right operand values and assigns the result to the left operand.
/=	Divide left operand value by right operand value and assign the result to the left operand.
%=	Get the modulus of left operand divide by right operand and assign resulted modulus to the left operand.

```
let x = 5, y = 10, z = 15;
x = y; //x would be 10
x += 1; //x would be 6
x -= 1; //x would be 4
x *= 5; //x would be 25
x /= 5; //x would be 1
x %= 2; //x would be 1
```

Ternary Operator

JavaScript provides a special operator called ternary operator `:?` that assigns a value to a variable based on some condition. This is the short form of the if else condition.

```
<condition> ? <value1> : <value2>;
```

The ternary operator starts with conditional expression followed by the `?` operator. The second part (after `?` and before `:`) will be executed if the condition turns out to be true. Suppose, the condition returns `false`, then the third part (after :) will be executed.

```
let a = 10, b = 5;
let c = a > b? a : b; // value of c would be 10
let d = a > b? b : a; // value of d would be 5
```

JavaScript Data Types

In JavaScript, you can assign different types of values (data) to a variable e.g. string, number, boolean, etc.

```
let myVariable = 1; // numeric value
myVariable = 'one'; // string value
myVariable = true; // Boolean value
```

In the above example, different types of values are assigned to the same variable to demonstrate the loosely typed characteristics of JavaScript. Here, `1` is the number type, `'one'` is the string type, and `true` is the boolean type.

JavaScript includes primitive and non-primitive data types as per the latest ECMAScript 5.1 specification.

Primitive Data Types

The primitive data types are the lowest level of the data value in JavaScript. The followings are primitive data types in JavaScript:

Data Type	Description

String	String is a textual content wrapped inside `' '` or `" "` or `` ` ` `` (tick sign). Example: 'Hello World!', "This is a string", etc.
Number	Number is a numeric value. Example: 100, 4521983, etc.
BigInt	BigInt is a numeric value in the arbitrary precision format. Example: 453889879865131n, 200n, etc.
Boolean	Boolean is a logical data type that has only two values, true or false.
Null	A null value denotes an absense of value. Example: `let str = null;`
Undefined	undefined is the default value of a variable that has not been assigned any value. Example: In the variable declaration, `var str;`, there is no value assigned to `str`. So, the type of `str` can be check using `typeof(str)` which will return `undefined`.

Structural Data Types

The structural data types contain some kind of structure with primitive data.

Data Type	Description

Object	An object holds multiple values in terms of properties and methods.
	
	Example:
	`let person = {`
	`firstName: "James",`
	`lastName: "Bond",`
	`age: 15`
	`};`

Date	The Date object represents date & time including days, months, years, hours, minutes, seconds, and milliseconds. Example: `let today = new Date("25 July 2021");`
Array	An array stores multiple values using special syntax. Example: `let nums = [1, 2, 3, 4];`

JavaScript Strings

In JavaScript, a string is a primitive data type that is used for textual data. JavaScript string must be enclosed in single quotes, double quotes, or backticks. The followings are string literals in JavaScript.

```
"Hello World"

'Hello World'

`Hello World`
```

The string literal can be assigned to a variable using the equal to `=` operator.

```
let str1 = "This is a double quoted string.";

let str2 = 'This is a single quoted string.';

let str3 = `This is a template string.`;
```

The template string (using backticks) is used when you want to include the value of a variable or expressions into a string. Use `${variable or expression}` inside backticks as shown below.

```
let amount = 1000, rate = 0.05, duration = 3;
let result = `Total Amount Payble: ${amount*(1 + rate*duration)}`;
```

The template string can be spanned in multiple lines which is not allowed with a single or double quoted string, as shown below.

```
let str1 = `This
            is
            multi-line
            string`;

/*let str2 = "This
            will
            give
            error"; */
```

JavaScript string can be treated like a character array. You can access a character in a string using square brackets `[index]` or using the `str.at(pos)` method.

```
let str = 'Hello World';

let ch1 = str[0] // H
let ch2 = str[1] // e
let ch3 = str.at(2) // l
let ch4 = str.at(3) // l
```

```
str[4] = "P"; //error
```

JavaScript strings can be accessed using a for loop, as shown below.

```
let str = 'Hello World';

for(let i =0; i< str.length; i++)
  console.log(str[i]);

for(let ch of str)
    console.log(ch);
```

Quotes Inside String

You can include single quotes in double-quoted string or include double quotes in a single quoted string. However, you cannot include a single quotes in single quoted string and double quotes in double-quoted string.

```
let str1 = "This is 'simple' string";

let str2 = 'This is "simple" string';

let str3 = `This is 'simple' and "easy" string`;
```

If you want to include the same quotes in a string value as surrounding quotes then use a backward slash (\) before the quotation mark inside the string value.

```
let str1 = "This is \"simple\" string";

let str2 = 'This is \'simple\' string';
```

String Concatenation

JavaScript string can be concatenated using the `+` operator or `string.concat()` method.

```
let str1 = 'Hello ';
let str2 = "World ";

let str3 = str1 + str2; //Hello World
let str4 = str1.concat(str2);//Hello World
```

String Objects

JavaScript allows you to create a string object using the `new` keyword, as shown below.

```
let str1 = new String(); //create string object
str1 = 'Hello World'; //assign value

// or

let str2 = new String('Hello World'); //create and assign value
```

String objects and string literals are different. The `typeof()` method will return the type of a variable. The following distinguished string and string objects.

```
let str1 = new String('Hello World');
let str2 = "Hello World";

typeof(str1); //"object"
typeof(str2); //"string"
```

Strings Comparison

Two strings can be compared using `<`, `>`, `==`, `===` operator, and `string.localeCompare(string)` method.

The mathematical operators `<` and `>` compare two strings and return a boolean (true or false) based on the order of the characters in the string.

The `==` operator compares the content of strings and `===` compares the reference equality of strings. The `localeCompare()` method compares two strings in the current locale. It returns `0` if strings are equal, else returns `1`.

```
console.log("a" < "b"); //true
console.log("b" < "a"); //false
console.log("Apple" == "Apple"); //true
console.log("Apple" == "apple"); //false
console.log("Apple" === "Apple"); //true
console.log("Apple" === "apple"); //false
console.log("Apple".localeCompare("Apple")); //0
console.log("Apple".localeCompare("apple")); //1
```

Note that the `===` operator compares the reference of strings objects and not the values.

```
let str1 = "Hello";
let str2 = 'Hello';
let str3 = new String('Hello');

console.log(str1 == str2); //true
console.log(str1 === str2);//true
console.log(str1 == str3); //true
console.log(str1 === str3);//false
```

JavaScript String Methods & Properties

JavaScript string (primitive or String object) includes default properties and methods which you can use for different purposes.

String Properties

Property	Description

length	Returns the length of the string.

charAt(position)	Returns the character at the specified position (in Number).
charCodeAt(position)	Returns a number indicating the Unicode value of the character at the given position (in Number).
concat([string,,])	Joins specified string literal values (specify multiple strings separated by comma) and returns a new string.
indexOf(SearchString, Position)	Returns the index of first occurrence of specified String starting from specified number index. Returns -1 if not found.9
lastIndexOf(SearchString, Position)	Returns the last occurrence index of specified SearchString, starting from specified position. Returns -1 if not found.
localeCompare(string,position)	Compares two strings in the current locale.
match(RegExp)	Search a string for a match using specified regular expression. Returns a matching array.
replace(searchValue, replaceValue)	Search specified string value and replace with specified replace Value string and return new string. Regular expression can also be used as searchValue.
search(RegExp)	Search for a match based on specified regular expression.

slice(startNumber, endNumber)	Extracts a section of a string based on specified starting and ending index and returns a new string.
split(separatorString, limitNumber)	Splits a String into an array of strings by separating the string into substrings based on specified separator. Regular expression can also be used as separator.
substr(start, length)	Returns the characters in a string from specified starting position through the specified number of characters (length).
substring(start, end)	Returns the characters in a string between start and end indexes.
toLocaleLowerCase()	Converts a string to lower case according to current locale.
toLocaleUpperCase()	Converts a sting to upper case according to current locale.
toLowerCase()	Returns lower case string value.
toString()	Returns the value of String object.
toUpperCase()	Returns upper case string value.
valueOf()	Returns the primitive value of the specified string object.

String Methods for Html

The following string methods convert the string as a HTML wrapper element.

Method	Description
anchor()	Creates an HTML anchor <a>element around string value.
big()	Wraps string in <big> element.
blink()	Wraps a string in <blink> tag.
bold()	Wraps string in <b> tag to make it bold in HTML.
fixed()	Wraps a string in <tt> tag.
fontcolor()	Wraps a string in a <font color="color"> tag.
fontsize()	Wraps a string in a <font size="size"> tag.
italics()	Wraps a string in <i> tag.
link()	Wraps a string in <a>tag where href attribute value is set to specified string.
small()	Wraps a string in a <small>tag.
strike()	Wraps a string in a <strike> tag.
sub()	Wraps a string in a <sub>tag
sup()	Wraps a string in a <sup>tag

JavaScript Numbers: Integer, Float, Binary, Exponential, Hexadecimal, Octal

The Number is a primitive data type used for positive or negative integer, float, binary, octal, hexadecimal, and exponential values in JavaScript.

The Number type in JavaScript is double-precision 64 bit binary format like double in C# and Java. It follows the international IEEE 754 standard.

The first character in a number type must be an integer value, and it must not be enclosed in quotation marks. The following example shows the variables having different types of numbers in JavaScript.

```
var num1 = 100; // integer
var num2 = -100; //negative integer
var num3 = 10.52; // float
var num4 = -10.52; //negative float
var num5 = 0xfff; // hexadecimal
var num6 = 256e-5; // exponential
var num7 = 030; // octal
var num8 = 0b0010001; // binary
```

Integers

Numbers can be positive or negative integers. However, integers are floating-point values in JavaScript. Integers value will be accurate up to 15 digits in JavaScript. Integers with 16 digits onwards will be

changed and rounded up or down; therefore, use BigInt for integers larger than 15 digits.

```
//16 digit integer
var int1 = 1234567890123456; //accurate

//17 digit integer
var int2 = 12345678901234569; //will be 12345678901234568

//16 digit integer
var int3 = 9999999999999998; //will be 9999999999999998

//16 digit integer, last digit 9
var int4 = 9999999999999999; //will be 10000000000000000
```

BigInt

The BigInt type is a numeric primitive type that can store integers with arbitrary precision. Use the BigInt for the large integers having more than 15 digits. Append `n` to the end of an integer to make it BigInt.

```
//16 digit integer
var int1 = 1234567890123459n; //will be 1234567890123459

//17 digit integer
var int2 = 12345678901234569n; //will be 12345678901234569
//20 digit integer
var int3 = 99999999999999999n; //will be 9999999999999999999
```

Floating-point Numbers

The floating-point numbers in JavaScript can only keep 17 decimal places of precision; beyond that, the value will be changed.

```
//17 decimal places
```

```
var f1 = 123456789012345.9; //accurate

//18 decimal places
var f2 = 1234567890123456.9; //will be 1234567890123457

//19 decimal places
var f3 = 1234567890123456.79; //will be 1234567890123456.8
```

Arithmetic operations on floating-point numbers in JavaScript are not always accurate. For example:

```
var f1 = 5.1 + 5.2; //will be 10.3
var f2 = 10.1 + 10.2; //will be 20.299999999999997
var f3 = (10.1*100 + 10.2*100)/100; //instead of 10.1 + 10.2
```

Arithmetic operation (except addition) of the numeric string will result in a number, as shown below.

```
var numStr1 = "5", numStr2 = "4";
var multiplication = numStr1 * numStr2; //returns20
var division = numStr1 / numStr2; //returns 1.25
var modulus = numStr1 % numStr2; //returns 1
```

Even if one of the values is a number, the result would be the same.

```
var num = 5, str = "4";
var multiplication = num * str; //returns 20
var division = num / str; //returns 1.25
var modulus = num % str; //returns 1
```

The `+` operator concatenates if any one value is a literal string.

```
var num = 5, str = "4";
var result = num + str; //returns "54"
```

Binary, Octal, Hexadecimal, Exponential

The binary numbers must start with `0b` or `0B` followed by 0 or 1.

The octal numbers must start with zero and the lower or upper letter 'O', `0o` or `0O`.

The Hexadecimal numbers must start with zero and the lower or upper letter 'X', `0x` or `0X`.

The exponential numbers should follow the `beN` format where `b` is a base integer or float number followed by `e` char, and `N` is an exponential power number.

```
var b = 0b100; // binary
var oct = 0o544; // octal
var hex = 0x123456789ABCDEF; // hexadecimal
var exp = 256e-5; // exponential
```

Number() Function in JavaScript

The `Number()` is a constructor function in JavaScript that converts values of other types to numbers.

```
var i = Number('100');
var f = Number('10.5');
var b = Number('0b100');
typeof(i); // returns number
typeof(f); // returns number
typeof(b); // returns number
```

By using the new operator with the `Number()` function will return an object which contains constants and methods for working with numbers.

```
var i = new Number('100');
var f = new Number('10.5');
var b = new Number('0b100');
typeof(i); // returns object
```

```
typeof(f); // returns object
typeof(b); // returns object
```

Compare Numbers

Be careful while comparing numbers using `==` or `===` operators. The `==` operator compares object references and not the values whereas the `===` operator compare values. The following example compares numbers created by different ways.

```
var num1 = new Number(100);
var num2 = Number('100');
var num3 = 100;
num1 == num2; // true
num1 === num2; // false
num2 == num3;//true
num2 === num3; // true
num1 == num3;//true
num1 === num3;//false
```

Number Properties

The Number type includes some default properties. JavaScript treats primitive values as objects, so all the properties and methods are applicable to both literal numbers and number objects.

The following table lists all the properties of Number type.

Property	Description
MAX_VALUE	Returns the maximum number value supported in JavaScript

MIN_VALUE	Returns the smallest number value supported in JavaScript
NEGATIVE_INFINITY	Returns negative infinity (-Infinity)
NaN	Represents a value that is not a number.
POSITIVE_INFINITY	Represents positive infinity (Infinity).

```
Number.MAX_VALUE; //1.7976931348623157e+308
Number.MIN_VALUE; //5e-324
Number.NEGATIVE_INFINITY; //-Infinity
Number.POSITIVE_INFINITY; //Infinity
Number.NaN;//NaN
```

Number Methods

The following table lists all the methods of Number type

Method	Description

toExponential(fractionDigits)	Returns exponential value as a string. Example: `var num = 100; num.toExponential(2); // returns '1.00e+2'`
toFixed(fractionDigits)	Returns string of decimal value of a number based on specified fractionDigits. Example: `var num = 100; num.toFixed(2); // returns '100.00'`
toLocaleString()	Returns a number as a string value according to a browser's locale settings. Example: `var num = 100; num.toLocaleString(); // returns '100'`
toPrecision(precisionNumber)	Returns number as a string with specified total digits. Example: `var num = 100; num.toPrecision(4); // returns '100.0'`
toString()	Returns the string representation of the number value. Example: `var num = 100; num.toString(); // returns '100'`
valueOf()	Returns the value of Number object. Example: `var num = new Number(100); num.valueOf(); // returns '100'`

JavaScript Booleans

The boolean (not Boolean) is a primitive data type in JavaScript. It can have only two values: true or false. It is useful in controlling program flow using conditional statements like if else, switch, while loop, etc.

The followings are boolean variables.

Example: boolean Variables

```
var YES = true;
var NO = false;
```

The following example demonstrates how a boolean value controls the program flow using the if condition.

Example: Boolean

```
var YES = true; var NO = false; if(YES) { alert("This code block will be executed"); } if(NO) { alert("This code block will not be executed"); }
```

The comparison expressions return boolean values to indicate whether the comparison is true or false. For example, the following expressions return boolean values.

```
var a = 10, b = 20;
var result = 1 > 2; // false
result = a < b; // true
result = a > b; // false
result = a + 20 > b + 5; // true
```

Boolean Function

JavaScript provides the `Boolean()` function that converts other types to a boolean type. The value specified as the first parameter will be converted to a boolean value. The `Boolean()` will return `true` for any non-empty, non-zero, object, or array.

Example: Boolean() Function

```
var a = 10, b = 20;
var b1 = Boolean('Hello'); // true
var b2 = Boolean('h'); // true
var b3 = Boolean(10); // true
var b4 = Boolean([]); // true
var b5 = Boolean(a + b); // true
```

If the first parameter is 0, -0, null, false, NaN, undefined, '' (empty string), or no parameter passed then the `Boolean()` function returns `false`.

```
var b1 = Boolean(''); // false
var b2 = Boolean(0); // false
var b3 = Boolean(null); // false
var a; var b4 = Boolean(a); // false
```

The new operator with the `Boolean()` function returns a `Boolean` object.

```
var bool = new Boolean(true); alert(bool); // true
```

Any boolean object, when passed in a conditional statement, will evaluate to `true`.

Example: Boolean Object in Condition

```
var bool = new Boolean(false);
```

```
if(bool){
alert('This will be executed.');
 }
```

Boolean vs boolean

The `new Boolean()` will return a Boolean object, whereas it returns a boolean without the new keyword. The boolean (lower case) is the primitive type, whereas Boolean (upper case) is an object in JavaScript. Use the `typeof` operator to check the types.

Example: Boolean vs boolean

```
var b1 = new Boolean(true); var b2 = true; typeof b1; // object
typeof b2; // boolean
```

Boolean Methods

Primitive or Boolean object includes following methods.

Method	Description
toLocaleString()	Returns string of boolean value in local browser environment. Example: `var result = (1 > 2); result.toLocaleString(); // returns "false"`
toString()	Returns a string of Boolean. Example: `var result = (1 > 2); result.toString(); // returns "false"`
valueOf()	Returns the value of the Boolean object. Example: `var result = (1 > 2); result.valueOf(); // returns false`

JavaScript Objects: Create Objects, Access Properties & Methods

Here you will learn objects, object literals, Object() constructor function, and access object in JavaScript.

You learned about primitive and structured data types in JavaScript. An object is a non-primitive, structured data type in JavaScript. Objects are same as variables in JavaScript, the only difference is that an object holds multiple values in terms of properties and methods.

In JavaScript, an object can be created in two ways:

1) using Object Literal/Initializer Syntax

2) using the `Object()` Constructor function with the new keyword. Objects created using any of these methods are the same.

The following example demonstrates creating objects using both ways.

```
var p1 = { name:"Steve" }; // object literal syntax
var p2 = new Object(); // Object() constructor function
p2.name = "Steve"; // property
```

Above, `p1` and `p2` are the names of objects. Objects can be declared same as variables using `var` or `let` keywords. The `p1` object is created using the object literal syntax (a short form of creating objects) with a property named `name`. The `p2` object is created by calling the `Object()`

constructor function with the `new` keyword. The `p2.name = "Steve";` attach a property `name` to `p2` object with a string value `"Steve"`.

Create Object using Object Literal Syntax

The object literal is a short form of creating an object. Define an object in the `{ }` brackets with key:value pairs separated by a comma. The key would be the name of the property and the value will be a literal value or a function.

```
var <object-name> = { key1: value1, key2: value2,...};
```

The following example demonstrates objects created using object literal syntax.

```
var emptyObject = {}; // object with no properties or methods
var person =
 {
firstName: "John" };
// object with single property
// object with single method
var message = {
     showMessage: function (val)
          {
               alert(val);
          }
}; // object with properties & method
var person =
{ firstName: "James",
lastName: "Bond", age: 15,
getFullName:


function () {
return
this.firstName + ' ' + this.lastName
```

```
}
 };
```

Note that the whole key-value pair must be declared. Declaring only a key without a value is invalid, as shown below.

```
var person = { firstName };
var person = { getFullName: };
```

Create Objects using Objects() Constructor

Another way of creating objects is using the `Object()` constructor function using the new keyword. Properties and methods can be declared using the dot notation `.property-name` or using the square brackets `["property-name"]`, as shown below.

```
var person = new Object(); // Attach properties and methods to person object
person.firstName = "James";
person["lastName"] = "Bond";
person.age = 25;
person.getFullName = function ()
 {
return this.firstName + ' ' + this.lastName;
 };
```

An object can have variables as properties or can have computed properties, as shown below.

```
var firstName = "James";
var lastName = "Bond";
var person = { firstName, lastName }
```

Access JavaScript Object Properties & Methods

An object's properties can be accessed using the dot notation `obj.property-name` or the square brackets `obj["property-name"]`. However, method can be invoked only using the dot notation with the parenthesis, `obj.method-name()`, as shown below.

```
var person =
{ firstName: "James",
lastName: "Bond",
age: 25,
getFullName: function ()
{ return this.firstName + ' ' + this.lastName } };
person.firstName; // returns James
person.lastName; // returns Bond
person["firstName"];// returns James
person["lastName"];// returns Bond
person. // calling getFullName();getFullName function
```

In the above example, the `person.firstName` access the `firstName` property of a `person` object. The `person["firstName"]` is another way of accessing a property. An object's methods can be called using () operator e.g. `person.getFullName()`. JavaScript engine will return the function definition if accessed method without the parenthesis.

Accessing undeclared properties of an object will return undefined. If you are not sure whether an object has a particular property or not, then use the `hasOwnProperty()` method before accessing them, as shown below.

```
var person = new Object();
person.firstName;//returns                                    undefined
if(person.hasOwnProperty("firstName")){
 person.firstName;
}
```

The properties and methods will be available only to an object where they are declared.

```
var p1 = new Object();
p1.firstName = "James";
p1.lastName = "Bond";
var p2 = new Object();
p2.firstName; // undefined
p2.lastName; //
undefined p3 = p1; // assigns object
p3.firstName; // James
p3.lastName; // Bond
p3.firstName = "Sachin"; // assigns new value
p3.lastName = "Tendulkar"; // assigns new value
```

Enumerate Object's Properties

Use the `for in` loop to enumerate an object, as shown below.

```
var person = new Object();
person.firstName = "James";
person.lastName = "Bond";
for(var prop in person){ alert(prop); // access property name
alert(person[prop]); // access property value };
```

Pass by Reference

Object in JavaScript passes by reference from one function to another.

```
function changeFirstName(per)
{ per.firstName = "Steve"; }
var person = {
 firstName : "Bill" };
changeFirstName(person)
```

```
person.firstName; // returns Steve
```

Nested Objects

An object can be a property of another object. It is called a nested object.

```
var person = {
firstName: "James",
lastName: "Bond",
age: 25,
address: {
     id: 1,
     country:"UK" }
     };
person.address.country; // returns "UK"
```

JavaScript object is a standalone entity that holds multiple values in terms of properties and methods.

Object property stores a literal value and method represents function.

An object can be created using object literal or object constructor syntax.

Object literal:

```
var person = {
    firstName: "James",
    lastName: "Bond",
    age: 25,
    getFullName: function () {
        return this.firstName + ' ' + this.lastName
        }
};
```

Object constructor:

```
var person = new Object();

person.firstName = "James";
person["lastName"] = "Bond";
person.age = 25;
person.getFullName = function () {
        return this.firstName + ' ' + this.lastName;
    };
```

Object properties and methods can be accessed using dot notation or [ ] bracket.

An object is passed by reference from one function to another.

An object can include another object as a property.

JavaScript Date: Create, Convert, Compare Dates in JavaScript

JavaScript provides Date object to work with date & time, including days, months, years, hours, minutes, seconds, and milliseconds.

Use the `Date()` function to get the string representation of the current date and time in JavaScript. Use the new keyword in JavaScript to get the Date object.

```
Date(); //Returns current date and time string //or
var currentDate = new Date();
//returns date object of current date and time
```

Create a date object by specifying different parameters in the `Date()` constructor function.

```
new Date()
new Date(value)
new Date(dateString)
new Date(year, monthIndex)
new Date(year, monthIndex, day)
new Date(year, monthIndex, day, hours)
new Date(year, monthIndex, day, hours, minutes)
new Date(year, monthIndex, day, hours, minutes, seconds)
new Date(year, monthIndex, day, hours, minutes, seconds,
milliseconds)
```

Parameters:

No Parameters: A date object will be set to the current date & time if no parameter is specified in the constructor.
value: An integer value representing the number of milliseconds since January 1, 1970, 00:00:00 UTC.
dateString: A string value that will be parsed using `Date.parse()` method.
year: An integer value to represent a year of a date. Numbers from 0 to 99 map to the years 1900 to 1999. All others are actual years.
monthIndex: An integer value to represent a month of a date. It starts with 0 for January till 11 for December
day: An integer value to represent day of the month.
hours: An integer value to represent the hour of a day between 0 to 23.
minutes: An integer value to represent the minute of a time segment.
seconds: An integer value to represent the second of a time segment.
milliseconds: An integer value to represent the millisecond of a time segment. Specify numeric milliseconds in the constructor to get the date and time elapsed from 1/1/1970.

In the following example, a date object is created by passing milliseconds in the `Date()` constructor function. So date will be calculated based on milliseconds elapsed from 1/1/1970.

```
var date1 = new Date(0); // Thu Jan 01 1970 05:30:00
var date2 = new Date(1000); // Thu Jan 01 1970 05:30:01
var date3 = new Date(5000); // Thu Jan 01 1970 05:30:05
```

The following example shows various formats of a date string that can be specified in a `Date()` constructor.

```
var date1 = new Date("3 march 2015");
var date2 = new Date("3 February, 2015");
var date3 = new Date("3rd February, 2015"); // invalid date
var date4 = new Date("2015 3 February");
var date5 = new Date("3 2015 February ");
var date6 = new Date("February 3 2015");
var date7 = new Date("February 2015 3");
var date8 = new Date("2 3 2015");
var date9 = new Date("3 march 2015 20:21:44");
```

You can use any valid separator in the date string to differentiate date segments.

```
var date1 = new Date("February 2015-3");
var date2 = new Date("February-2015-3");
var date3 = new Date("February-2015-3");
var date4 = new Date("February,2015-3");
var date5 = new Date("February,2015,3");
var date6 = new Date("February*2015,3");
var date7 = new Date("February$2015$3");
var date8 = new Date("3-2-2015"); // MM-dd-YYYY
var date9 = new Date("3/2/2015"); // MM-dd-YYYY
```

Specify seven numeric values to create a date object with the specified year, month and optionally date, hours, minutes, seconds and milliseconds.

```
var date1 = new Date(2021, 2, 3); // Mon Feb 03 2021
```

```
var date2 = new Date(2021, 2, 3, 10); // Mon Feb 03 2021 10:00
var date3 = new Date(2021, 2, 3, 10, 30); // Mon Feb 03 2021
10:30
var date4 = new Date(2021, 2, 3, 10, 30, 50); // Mon Feb 03 2021
10:30:50
var date5 = new Date(2021, 2, 3, 10, 30, 50, 800); // Mon Feb 03
2021 10:30:50
```

Date Formats

JavaScript supports ISO 8601 date format by default - `YYYY-MM-DDTHH:mm:ss.sssZ`

```
var dt = new Date('2015-02-10T10:12:50.5000z');
```

Convert Date Formats

Use different Date methods to convert a date from one format to another format, e.g., to Universal Time, GMT, or local time format.

The following example demonstrates `ToUTCString()`, `ToGMTString()`, `ToLocalDateString()`, and `ToTimeString()` methods to convert date into respective formats.

```
var date = new Date('2015-02-10T10:12:50.5000z');
date; 'Default format:'
date.toDateString();'Tue          Feb          10          2015'
date.toLocaleDateString();'2/10/2015'
date.toGMTString(); 'GMT format'
date.toISOString();                    '2015-02-10T10:12:50.500Z'
date.toLocaleString();'Local         date         Format         '
date.toLocaleTimeString();      'Locale      time      format      '
date.toString('YYYY-MM-dd');   'Tue   Feb   10   2015   15:42:50'
date.toTimeString(); '15:42:50'
date.toUTCString(); 'UTC format '
```

To get date string in formats other than the ones listed above, you need to manually form the date string using different date object methods. The following example converts a date string to DD-MM-YYYY format.

```
var date = new Date('4-1-2015'); // M-D-YYYY
var d = date.getDate();
var m = date.getMonth() + 1;
var y = date.getFullYear();
var dateString = (d <= 9 ? '0' + d : d) + '-' + (m <= 9 ? '0' +
m : m) + '-' + y;
```

Compare Dates in JavaScript

Use comparison operators to compare two date objects.

```
var date1 = new Date('4-1-2015');
var date2 = new Date('4-2-2015');
if (date1 > date2)
     alert(date1 + ' is greater than ' + date2);
else
     (date1 < date2 ) alert(date1 + ' is less than ' + date2);
```

Date Methods Reference

The following table lists all the get methods of Date object.

Method	Description
getDate()	Returns numeric day (1 - 31) of the specified date.
getDay()	Returns the day of the week (0 - 6) for the specified date.
getFullYear()	Returns four digit year of the specified date.
getHours()	Returns the hour (0 - 23) in the specified date.
getMilliseconds()	Returns the milliseconds (0 - 999) in the specified date.
getMinutes()	Returns the minutes (0 - 59) in the specified date.
getMonth()	Returns the month (0 - 11) in the specified date.
getSeconds()	Returns the seconds (0 - 59) in the specified date.
getTime()	Returns the milliseconds as number since January 1, 1970, 00:00:00 UTC.
getTimezoneOffset()	Returns the time zone offset in minutes for the current locale.
getUTCDate()	Returns the day (1 - 31) of the month of the specified date as per UTC time zone.
getUTCDay()	Returns the day (0 - 6) of the week of the specified date as per UTC timezone.

getUTCFullYear()	Returns the four digits year of the specified date as per UTC time zone.
getUTCHours()	Returns the hours (0 - 23) of the specified date as per UTC time zone.
getUTCMilliseconds()	Returns the milliseconds (0 - 999) of the specified date as per UTC time zone.
getUTCMinutes()	Returns the minutes (0 - 59) of the specified date as per UTC time zone.
getUTCMonth()	Returns the month (0 - 11) of the specified date as per UTC time zone.
getUTCSeconds()	Returns the seconds (0 - 59) of the specified date as per UTC time zone.
getYear()	Returns the no of years of the specified date since 1990. *This method is Deprecated*

The following table lists all the set methods of Date object.

Method	Description
setDate()	Sets the day as number in the date object.
setFullYear()	Sets the four digit full year as number in the date object. Optionally set month and date.
setHours()	Sets the hours as number in the date object. Optionally set minutes, seconds and milliseconds.
setMilliseconds()	Sets the milliseconds as number in the date object.
setMinutes()	Sets the minutes as number in the date object. Optionally set seconds & milliseconds.
setMonth()	Sets the month as number in the date object. Optionally set date.
setSeconds()	Sets the seconds as number in the date object. Optionally set milliseconds.
setTime()	Sets the time as number in the Date object since January 1, 1970, 00:00:00 UTC.
setUTCDate()	Sets the day in the date object as per UTC time zone.
setUTCFullYear()	Sets the full year in the date object as per UTC time zone
setUTCHours()	Sets the hour in the date object as per UTC time zone
setUTCMilliseconds()	Sets the milliseconds in the date object as per UTC time zone
setUTCMinutes()	Sets the minutes in the date object as per UTC time zone

setUTCMonth()	Sets the month in the date object as per UTC time zone
setUTCSeconds()	Sets the seconds in the date object as per UTC time zone
setYear()	Sets the year in the date object. *This method is Deprecated*

toDateString()	Returns the date segment from the specified date, excludes time.
toGMTString()	Returns a date string in GMT time zone.
toLocaleDateString()	Returns the date segment of the specified date using the current locale.
toLocaleFormat()	Returns a date string in default format.
toLocaleString()	Returns a date string using a current locale format.
toLocaleTimeString()	Returns the time segment of the specified Date as a string.
toString()	Returns a string for the specified Date object.
toTimeString()	Returns the time segment as a string from the specified date object.
toUTCString()	Returns a string as per UTC time zone.
valueOf()	Returns the primitive value of a Date object.

JavaScript Arrays: Create, Access, Add & Remove Elements

We have learned that a variable can hold only one value. We cannot assign multiple values to a single variable. JavaScript array is a special type of variable, which can store multiple values using a special syntax.

The following declares an array with five numeric values.

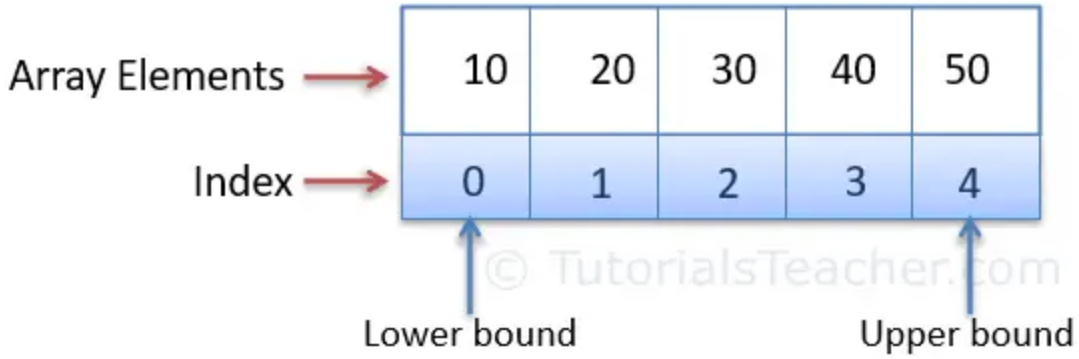
```
let numArr = [10, 20, 30, 40, 50];
```

In the above array, `numArr` is the name of an array variable. Multiple values are assigned to it by separating them using a comma inside square brackets as `[10, 20, 30, 40, 50]`. Thus, the `numArr` variable stores five numeric values. The `numArr` array is created using the literal syntax and it is the preferred way of creating arrays.

Another way of creating arrays is using the `Array()` constructor, as shown below.

```
let numArr = new Array(10, 20, 30, 40, 50);
```

Every value is associated with a numeric index starting with 0. The following figure illustrates how an array stores values.



JavaScript Array Representation

The following are some more examples of arrays that store different types of data.

Example: Array Literal Syntax

```
let stringArray = ["one", "two", "three"];

let numericArray = [1, 2, 3, 4];

let decimalArray = [1.1, 1.2, 1.3];

let booleanArray = [true, false, false, true];
```

It is not required to store the same type of values in an array. It can store values of different types as well.

```
let data = [1, "Steve", "DC", true, 255000, 5.5];
```

Get Size of an Array

Use the `length` property to get the total number of elements in an array. It changes as and when you add or remove elements from the array.

Example: Get Array Size

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];
console.log(cities.length); //4

cities[4] = "Delhi";
console.log(cities.length); //5
```

Accessing Array Elements

Array elements (values) can be accessed using an index. Specify an index in square brackets with the array name to access the element at a particular index like `arrayName[index]`. Note that the index of an array starts from zero.

Example: Accessing Array Elements

```
let numArr = [10, 20, 30, 40, 50];
console.log(numArr[0]); // 10
console.log(numArr[1]); // 20
console.log(numArr[2]); // 30
console.log(numArr[3]); // 40
console.log(numArr[4]); // 50

let cities = ["Mumbai", "New York", "Paris", "Sydney"];

console.log(cities[0]); // "Mumbai"
console.log(cities[1]); // "New York"
```

```
console.log(cities[2]); // "Paris"
console.log(cities[3]); // "Sydney"

//accessing element from nonexistance index
console.log(cities[4]); // undefined
```

For the new browsers, you can use the `arr.at(pos)` method to get the element from the specified index. This is the same as `arr[index]` except that the `at()` returns an element from the last element if the specified index is negative.

Example: Accessing Array using at()

```
let numArr = [10, 20, 30, 40, 50];
console.log(numArr.at(0)); // 10
console.log(numArr.at(1)); // 20
console.log(numArr.at(2)); // 30
console.log(numArr.at(3)); // 40
console.log(numArr.at(4)); // 50
console.log(numArr.at(5)); // undefined

//passing negative index
console.log(numArr.at(-1)); // 50
console.log(numArr.at(-2)); // 40
console.log(numArr.at(-3)); // 30
console.log(numArr.at(-4)); // 20
console.log(numArr.at(-5)); // 10
console.log(numArr.at(-6)); // undefined
```

You can iterate an array using `Array.forEach()`, for, for-of, and for-in loop, as shown below.

Example: Accessing Array Elements

```
let numArr = [10, 20, 30, 40, 50];

numArr.forEach(i =>  console.log(i)); //prints all elements

for(let i=0; i<numArr.length; i++)
  console.log(numArr[i]);

for(let i of numArr)
  console.log(i);

for(let i in numArr)
  console.log(numArr[i]);
```

Update Array Elements

You can update the elements of an array at a particular index using `arrayName[index] = new_value` syntax.

Example: Update Array Elements

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];

cities[0] = "Delhi";
cities[1] = "Los angeles";

console.log(cities); //["Delhi", "Los angeles", "Paris",
"Sydney"]
```

Adding New Elements

You can add new elements using `arrayName[index] = new_value` syntax. Just make sure that the index is greater than the last index. If you specify an existing index then it will update the value.

Example: Add Array Elements

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];

cities[4] = "Delhi"; //add new element at last
console.log(cities); //["Mumbai", "New York", "Paris", "Sydney",
"Delhi"]

cities[cities.length] = "London";//use length property to
specify last index
console.log(cities); //["Mumbai", "New York", "Paris", "Sydney",
"Delhi", "London"]

cities[9] = "Pune";
console.log(cities); //["Mumbai", "New York", "Paris", "Sydney",
"Delhi", "Londen", undefined, undefined, undefined, "Pune"]
```

In the above example, `cities[9] = "Pune"` adds `"Pune"` at 9th index and all other non-declared indexes as undefined.

The recommended way of adding elements at the end is using the `push()` method. It adds an element at the end of an array.

Example: Add Element At Last using push()

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];

cities.push("Delhi"); //add new element at last
```

```
console.log(cities); //["Mumbai", "New York", "Paris", "Sydney",
"Delhi"]
```

Use the `unshift()` method to add an element to the beginning of an array.

Example: Add Element using unshift()

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];

cities.unshift("Delhi"); //adds new element at the beginning
console.log(cities); //["Delhi", "Mumbai", "New York", "Paris",
"Sydney"]

cities.unshift("London", "Pune"); //adds new element at the
beginning
console.log(cities); //["London", "Pune", "Delhi", "Mumbai",
"New York", "Paris", "Sydney"]
```

Remove Array Elements

The `pop()` method returns the last element and removes it from the array.

Example: Remove Last Element

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];

let removedCity = cities.pop();
//returns and removes the last element
console.log(cities);
//["Mumbai", "New York", "Paris"]
```

The `shift()` method returns the first element and removes it from the array.

Example: Remove First Element

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];

let removedCity = cities.shift(); //returns first element and
removes it from array
console.log(cities); //["New York", "Paris", "Sydney"]
```

You cannot remove middle elements from an array. You will have to create a new array from an existing array without the element you do not want, as shown below.

Example: Remove Middle Elements

```
let cities = ["Mumbai", "New York", "Paris", "Sydney"];
let cityToBeRemoved = "Paris";

let mycities = cities.filter(function(item) {
    return item !== cityToBeRemoved
})

console.log(mycities); //["Mumbai", "New York", "Sydney"]
console.log(cities); //["Mumbai", "New York", "Paris", "Sydney"
```

Array Methods Reference

The following table lists all the Array methods.

Method	Description
concat()	Returns new array by combining values of an array that is specified as parameter with existing array values.
every()	Returns true or false if every element in the specified array satisfies a condition specified in the callback function. Returns false even if single element does not satisfy the condition.
filter()	Returns a new array with all the elements that satisfy a condition specified in the callback function.
forEach()	Executes a callback function for each elements of an array.
indexOf()	Returns the index of the first occurrence of the specified element in the array, or -1 if it is not found.
join()	Returns string of all the elements separated by the specified separator
lastIndexOf()	Returns the index of the last occurrence of the specified element in the array, or -1 if it is not found.
map()	Creates a new array with the results of calling a provided function on every element in this array.
pop()	Removes the last element from an array and returns that element.
push()	Adds one or more elements at the end of an array and returns the new length of the array.

reduce()	Pass two elements simultaneously in the callback function (till it reaches the last element) and returns a single value.
reduceRight()	Pass two elements simultaneously in the callback function from right-to-left (till it reaches the last element) and returns a single value.
reverse()	Reverses the elements of an array. Element at last index will be first and element at 0 index will be last.
shift()	Removes the first element from an array and returns that element.
slice()	Returns a new array with specified start to end elements.
some()	Returns true if at least one element in this array satisfies the condition in the callback function.
sort()	Sorts the elements of an array.
splice()	Adds and/or removes elements from an array.
toString()	Returns a string representing the array and its elements.
unshift()	Adds one or more elements to the front of an array and returns the new length of the array.

Difference between null and undefined in JavaScript

What is a null?

A null means the absence of a value. You assign a null to a variable with the intention that currently this variable does not have any value but it will have later on. It is like a placeholder for a value. The type of null is the object.

Example: null

```
let num = null;

console.log(num); // null

console.log(typeof num); // "object"
```

Sometimes, null variables are the result of erroneous code. For example, if you try to find an HTML element using `document.getElelementByID()` with the wrong id, then it will return null. So it is recommended to check for null before doing something with that element.

Example: null

```
var saveButton = document.getElementById("save");



if (saveButton !== null)

    saveButton.submit();
```

What is undefined?

A variable is undefined when you haven't assigned any value yet, not even a null.

Example: undefined Variable

```
let num;

console.log(num);//"undefined"
```

Generally, variables are undefined when you forgot to assign values or change existing code. For example, consider the following `Greet()` function that returns a string.

```
function Greet(){

    return "Hi";

}

let str = Greet();//"Hi"
```

Now, suppose somebody changes the function as below. So now, `str` will be undefined.

```
function Greet(){

    alert("Hi");

}

let str = Greet();//undefined
```

Thus, undefined variables are the result of some code problems.

Difference between null and undefined

You must explicitly assign a null to a variable. A variable has undefined when no value assigned to it.

Example: null and undefined Variables

```
let num1 = null;

let num2;



console.log(num1);//null

console.log(num2); //undefined
```

The `''` is not the same as null or undefined.

```
let str = '';



console.log(typeof str);//string

console.log(str === null); //false

console.log(str === undefined); //false
```

The type of null variable is object whereas the type of undefined variable is "undefined".

Example: Types

```
let num1 = null;

let num2;


console.log(typeof num1);//"object"

console.log(typeof num2); //"undefined"
```

Use the `===` operator to check whether a variable is null or undefined. The `==` operator gives the wrong result.

Example: Comparison using === and ==

```
let num1 = null;

let num2;


console.log(num1 == null); //true

console.log(num2 == undefined);//true


console.log(num1 == undefined);//true (incorrect)

console.log(num2 == null);//true (incorrect)


console.log(num1 === null); //true

console.log(num2 === undefined);//true
```

```
console.log(num1 === undefined);//false

console.log(num2 === null);//false



console.log(num1 == num2);//true (incorrect)

console.log(num1 === num2);//false
```

The null and undefined variables are falsy to if-statements and ternary operators.

Example: Null and undefined with if-statements

```
let num1 = null;

let num2;



if(num1)

{

    console.log(num1);

}

else

{

    console.log("num1 is null");
```

```
}


if(num2)
{
    console.log(num2);
}
else
{
    console.log("num2 is undefined");
}
```

A null variable treated as 0 in an numeric expression whereas undefined variable will be NaN.

Example: With Numeric Expresion

```
let num1 = null;
let num2;


console.log(num1 + 10);//10
console.log(num2 + 10); //NaN
```

It will give wrong result when concatenated with string.

Example: With String Values

```
let num1 = null;

let num2;



console.log(num1 + " Hello");//"null Hello"

console.log(num2 + " Hello"); //"undefined Hello"
```

Functions in JavaScript

Functions are the basic building block of JavaScript. Functions allow us to encapsulate a block of code and reuse it multiple times.

Functions make JavaScript code more readable, organized, reusable, and maintain

```
function <function-name>(arg1, arg2, arg3,...)

{

    //write function code here

};
```

In JavaScript, a function can be defined using the `function` keyword, followed by the name of a function and parentheses. Optionally, a list of input parameters can be included within the parentheses. The code block that needs to be executed when the function is called is written within curly braces.

Defining a Function in JavaScript

The following defines a function named `greet` that will display an alert box.

Example: Define a Function

```
function greet() {

    alert("Hello World!");

}
```

The above `greet()` function does not include any input parameters. It contains a single statement that displays an alert message.

Now, you can call or invoke the `greet` function by using the function name followed by the `()` operator, as shown below. When you call a function, JavaScript will execute the codes written inside the calling function.

```
greet();
```

Function Parameters

You can pass values to a function using parameters. A function can have one or more parameters, and the values will be passed by the calling code.

Example: Function Parameters

```
function greet(firstName, lastName) {
```

```
    alert("Hello " + firstName + " " + lastName);
}


greet("Steve", "Jobs");
```

JavaScript is a dynamic type scripting language, so a function parameter can have a value of any data type.

Example: Function Parameters

```
function greet(firstName, lastName) {
    alert("Hello " + firstName + " " + lastName);
}


greet("Bill", "Gates");
greet(100, 200);
```

You can pass fewer or more arguments while calling a function. If you pass fewer arguments then the rest of the parameters will become undefined. If you pass more arguments then additional arguments will be ignored.

Example: Function Parameters

```
function greet(firstName, lastName) {
    alert("Hello " + firstName + " " + lastName);
```

```
}


greet("Steve", "Jobs", "Mr."); // display Hello Steve Jobs

greet("Bill"); // display Hello Bill undefined

greet(); // display Hello undefined undefined
```

You can also use the built-in arguments object to access parameters inside a function.

Return a Value From a Function

A function can return a value to the calling code using the `return` keyword followed by a variable or a value.

The following returns a number `10`.

Example: Return a value of a Function

```
function getNumber() {

    return 10;

};


let result = getNumber();

console.log(result); //output: 10
```

Typically, a function returns some calculated value using paramters or an expression from a function. For example, the following `sum` function

adds two parameters values using the `+` operator and returns the result of an expression.

Example: Return value from a Function

```
function Sum(num1, num2) {

    return num1 + num2;

};



var result = Sum(10,20); // returns 30
```

A function can return another function in JavaScript.

Example: Function Returning a Function

```
function multiple(x) {


    function fn(y)

    {

        return x * y;

    }


    return fn;

}
```

```
var triple = multiple(3);

triple(2); // returns 6

triple(3); // returns 9
```

Function Expression

A function expression in JavaScript is a function that is stored as a value, and can be assigned to a variable or passed as an argument to another function.

Example: Function Expression

```
var add = function (num1, num2) {

    return num1 + num2;

};

var result = add(10, 20);//returns 30
```

Anonymous Function

In JavaScript, you can also create anonymous functions, which are functions without a name. Anonymous functions are often used as arguments to other functions.

Anonymous functions are typically used in functional programming e.g. callback function, creating closure or immediately invoked function expression.

Example: Anonymous Function

```
let numbers = [10, 20, 30, 40, 50];


let squareNumbers = numbers.map(function(number) {
  return number*number;
});
document.write(squareNumbers);
```

Arrow Functions

Arrow functions are a shorthand syntax for defining anonymous functions in JavaScript. They have compact syntax compared to anonymous functions. However, they do not have their own `this` value.

Example: Arrow Function

```
let square = num => num * num;
let result = square(5);
console.log(result); //25
```

Nested Functions

In JavaScript, a function can have one or more inner functions. These nested functions are in the scope of outer function. Inner function can access variables and parameters of outer function. However, outer function cannot access variables defined inside inner functions.

Example: Nested Functions

```
function greet(firstName){

 function SayHello() {

        alert("Hello " + firstName);

    }

    return SayHello();

}

greet("Steve");
```

JavaScript if...else Statements

JavaScript includes if/else conditional statements to control the program flow, similar to other programming languages.

JavaScript includes following forms of if-else statements:

1. if Statement
2. if else Statement
3. else if Statement

if Statement

Use if conditional statement if you want to execute something based on some condition.

```
if(boolean expression)
```

```
{

    // code to be executed if condition is true

}
if( 1 > 0) { alert("1 is greater than 0"); }
if( 1 < 0) { alert("1 is less than 0"); }
```

In the above example, the first if statement contains 1 > 0 as conditional expression. The conditional expression 1 > 0 will be evaluated to true, so an alert message "1 is greater than 0" will be displayed, whereas conditional expression in second if statement will be evaluated to false, so "1 is less than 0" alert message will not be displayed.

In the same way, you can use variables in a conditional expression.

```
var mySal = 1000;
var yourSal = 500;
if( mySal > yourSal) {
     alert("My Salary is greater than your salary");
      }
```

Use comparison operators carefully when writing conditional expression. For example, == and === is different.

```
     if(1=="1") {
     alert("== operator does not consider types of operands");
     }
     if(1==="1") {
     alert("=== operator considers types of operands");
     }
```

else condition

Use else statement when you want to execute the code every time when if condition evaluates to false.

The else statement must follow if or else if statement. Multiple else block is NOT allowed.

```
if(condition expression)
{
    //Execute this code..
}
else{
    //Execute this code..
}
var mySal = 500;
var yourSal = 1000;
if( mySal > yourSal)
{
alert("My Salary is greater than your salary");
 }
else
{
alert("My Salary is less than or equal to your salary");
 }
```

else if condition

Use "else if" condition when you want to apply second level condition after if statement.

```
if(condition expression)
{
    //Execute this code block
}
else if(condition expression){
    //Execute this code block
}
var mySal = 500;
var yourSal = 1000;
```

```
if( mySal > yourSal)
{
alert("My Salary is greater than your salary");
}
else
if(mySal < yourSal)
{
alert("My Salary is less than your salary");
}
```

JavaScript allows multiple else if statements also.

```
var mySal = 500;
var yourSal = 1000;
if( mySal > yourSal)
{
alert("My Salary is greater than your salary");
}
else
if(mySal < yourSal)
{
alert("My Salary is less than your salary");
}
else
if(mySal == yourSal)
{
alert("My Salary is equal to your salary");
}
```

Use if-else conditional statements to control the program flow.

JavaScript includes three forms of if condition: if condition, if else condition and else if condition.

The if condition must have conditional expression in brackets () followed by single statement or code block wrapped with { }.

'else if' statement must be placed after if condition. It can be used multiple times.

'else' condition must be placed only once at the end. It must come after if or else if statement.

JavaScript switch

The switch is a conditional statement like if statement. Switch is useful when you want to execute one of the multiple code blocks based on the return value of a specified expression.

```
switch(expression or literal value){
    case 1:
        //code to be executed
      break;
    case 2:
        //code to be executed
        break;
    case n:
        //code to be executed
        break;
    default:
        //default code to be executed
        //if none of the above case executed
}
```

As per the above syntax, switch statement contains an expression or literal value. An expression will return a value when evaluated. The switch can includes multiple cases where each case represents a particular value. Code under particular case will be executed when case value is equal to the return value of switch expression. If none of the cases match with switch expression value then the default case will be executed.

```
var a = 3;
switch (a)
{
     case 1: alert('case 1 executed');
```

```
      break;
      case 2: alert("case 2 executed");
      break;
      case 3: alert("case 3 executed");
      break;
      case 4: alert("case 4 executed");
      break;
      default: alert("default case executed");
}
```

In the above example, switch statement contains a literal value as expression. So, the case that matches a literal value will be executed, case 3 in the above example.

The switch statement can also include an expression. A case that matches the result of an expression will be executed.

```
var a = 3;
switch (a/3)
{
      case 1: alert("case 1 executed");
      break;
      case 2: alert("case 2 executed");
      break;
      case 3: alert("case 3 executed");
      break;
      case 4: alert("case 4 executed");
      break;
      default: alert("default case executed");
}
```

In the above example, switch statement includes an expression a/3, which will return 1 (because a = 3). So, case 1 will be executed in the above example.

The switch can also contain string type expression.

```
var str = "bill";
```

```
switch (str)
{
     case "steve": alert("This is Steve");
     case "bill": alert("This is Bill");
     break; case "john": alert("This is John");
     break;
     default: alert("Unknown Person");
     break;
}
```

Multiple cases can be combined in a switch statement.

```
var a = 2;
switch (a)
{
     case 1:
     case 2:
     case 3:
     alert("case 1, 2, 3 executed");
     break;
     case 4: alert("case 4 executed");
     break;
     default: alert("default case executed");
}
```

The switch is a conditional statement like if statement.

A switch statement includes literal value or is expression based

A switch statement includes multiple cases that include code blocks to execute.

A break keyword is used to stop the execution of case block.

A switch case can be combined to execute same code block for multiple cases.

JavaScript for Loop

JavaScript includes for loop like Java or C#. Use for loop to execute code repeatedly.

```
for(initializer; condition; iteration)

{
    // Code to be executed
}
```

The for loop requires following three parts.

Initializer: Initialize a counter variable to start with
Condition: specify a condition that must evaluate to true for next iteration
Iteration: increase or decrease counter

```
for (var i = 0; i < 5; i++) { console.log(i); }
```

Output:

```
0 1 2 3 4
```

In the above example, var i = 0 is an initializer statement where we declare a variable i with value 0. The second part, i < 5 is a condition where it checks whether i is less than 5 or not. The third part, i++ is iteration statement where we use ++ operator to increase the value of i to 1. All these three parts are separated by semicolon ;.

The for loop can also be used to get the values for an array.

```
var arr = [10, 11, 12, 13, 14];
for (var i = 0; i < 5; i++) { console.log(arr[i]); }
```

Output:

```
10 11 12 13 14
```

Please note that it is not mandatory to specify an initializer, condition and increment expression into bracket. You can specify initializer before starting for loop. The condition and increment statements can be included inside the block.

```
var arr = [10, 11, 12, 13, 14]; var i = 0; for (; ;) { if (i >= 5) break; console.log(arr[i]); i++; }
```

Output:
```
10 11 12 13 14
```

Learn about while loop in the next section.

JavaScript for loop is used to execute code repeatedly.

for loop includes three parts: initialization, condition and iteration.
e.g.`for(initializer; condition; iteration){ ... }`

The code block can be wrapped with { } brackets.

An initializer can be specified before starting for loop. The condition and increment statements can be included inside the block.

JavaScript - While Loop

JavaScript includes while loop to execute code repeatedly till it satisfies a specified condition. Unlike for loop, while loop only requires condition expression.

```
while(condition expression)

{    /* code to be executed    till the specified condition is true */

}
```

```
var i =0;
while(i < 5)
{
console.log(i);
i++;
 }
```

Output:
```
0 1 2 3 4
```

> Make sure condition expression is appropriate and include increment or decrement counter variables inside the while block to avoid infinite loop.

As you can see in the above example, while loop will execute the code block till i < 5 condition turns out to be false. Initialization statement for a counter variable must be specified before starting while loop and increment of counter must be inside while block.

do while

JavaScript includes another flavour of while loop, that is do-while loop. The do-while loop is similar to while loop the only difference is it evaluates condition expression after the execution of code block. So do-while loop will execute the code block at least once.

```
do{

   //code to be executed

}while(condition expression)
```

```
var i = 0; do{ alert(i); i++; } while(i < 5)
```

Output:
```
0 1 2 3 4
```

The following example shows that do-while loop will execute a code block even if the condition turns out to be false in the first iteration.

Example: do-while loop

```
var i =0; do{ alert(i); i++; } while(i > 1)
```

Output:

```
0
```

JavaScript while loop & do-while loop execute the code block repeatedly till conditional expression returns true.

do-while loop executes the code at least once even if condition returns false.

Scope in JavaScript

Scope in JavaScript defines accessibility of variables, objects and functions.

There are two types of scope in JavaScript.

1. Global scope
2. Local scope

Global Scope

Variables declared outside of any function become global variables. Global variables can be accessed and modified from any function.

Example: Global Variable

```
<script> var userName = "Bill"; function modifyUserName() { userName = "Steve"; }; function showUserName() { alert(userName); }; alert(userName); // display Bill modifyUserName(); showUserName();// display Steve </script>
```

In the above example, the variable userName becomes a global variable because it is declared outside of any function. A modifyUserName() function modifies userName as userName is a global variable and can be accessed inside any function. The same way, showUserName() function displays current value of userName variable. Changing value of global variable in any function will reflect throughout the program.

Please note that variables declared inside a function without var keyword also become global variables.

Example: Global Variable

```
<script> function createUserName() { userName = "Bill"; } function modifyUserName() { if(userName) userName = "Steve"; }; function showUserName() { alert(userName); } createUserName(); showUserName(); // Bill modifyUserName(); showUserName(); // Steve </script>
```

In the above example, variable userName is declared without var keyword inside createUserName(), so it becomes global variable automatically after calling createUserName() for the first time.

A userName variable will become global variable only after createUserName() is called at least once. Calling showUserName() before createUserName() will throw an exception "userName is not defined".

Local Scope

Variables declared inside any function with var keyword are called local variables. Local variables cannot be accessed or modified outside the function declaration.

Example: Local Scope

```
<script> function createUserName() { var userName = "Bill"; } function showUserName() { alert(userName); } createUserName(); showUserName(); // throws error: userName is not defined </script>
```

Function parameters are considered as local variables.

In the above example, userName is local to createUserName() function. It cannot be accessed in showUserName() function or any other functions. It will throw an error if you try to access a variable which is not in the local or global scope. Use try catch block for exception handling.

Some tips..

If local variable and global variable have same name then changing value of one variable does not affect on the value of another variable.

Example: Scope

```
var userName = "Bill"; function ShowUserName() { var userName = "Steve"; alert(userName); // "Steve" } ShowUserName(); alert(userName); // Bill
```

JavaScript does not allow block level scope inside { }. For example, variables defined in if block can be accessed outside if block, inside a function.

Example: No Block Level Scope

```
Function NoBlockLevelScope(){ if (1 > 0) { var myVar = 22; } alert(myVar); } NoBlockLevelScope();
```

JavaScript has global scope and local scope.

Variables declared and initialized outside any function become global variables.

Variables declared and initialized inside function becomes local variables to that function.

Variables declared without var keyword inside any function becomes global variables automatically.

Global variables can be accessed and modified anywhere in the program.

Local variables cannot be accessed outside the function declaration.

Global variable and local variable can have same name without affecting each other.

JavaScript does not allow block level scope inside { } brackets.

JavaScript eval

eval() is a global function in JavaScript that evaluates a specified string as JavaScript code and executes it.

Example: eval

```
eval("alert('this is executed by eval()')");
```

The eval() function can also call the function and get the result as shown below.

Example: eval

```
var result;
function Sum(val1, val2)
{
return val1 + val2;
}
eval("result = Sum(5, 5);");
alert(result);
```

eval can convert string to JSON object.

Example: eval with JSON Object

```
var str = '({"firstName":"Bill","lastName":"Gates"})';
var obj = eval(str);
obj.firstName; // Bill
```

Error handling in JavaScript

JavaScript is a loosely-typed language. It does not give compile-time errors. So some times you will get a runtime error for accessing an undefined variable or calling undefined function etc.

try catch block does not handle syntax errors.

JavaScript provides error-handling mechanism to catch runtime errors using try-catch-finally block, similar to other languages like Java or C#

```
try

{

    // code that may throw an error

}
```

```
catch(ex)

{

    // code to be executed if an error occurs

}

finally{

    // code to be executed regardless of an error occurs or not

}
```

try: wrap suspicious code that may throw an error in try block.
catch: write code to do something in catch block when an error occurs. The catch block can have parameters that will give you error information. Generally catch block is used to log an error or display specific messages to the user.
finally: code in the finally block will always be executed regardless of the occurrence of an error. The finally block can be used to complete the remaining task or reset variables that might have changed before error occurred in try block.

Let's look at simple error handling examples.

Example: Error Handling in JS

```
try {
var result = Sum(10, 20);
// Sum is not defined yet }
 catch(ex)
{ document.getElementById("errorMessage").innerHTML = ex; }
```

In the above example, we are calling function Sum, which is not defined yet. So, try block will throw an error which will be handled by catch block. Ex includes error message that can be displayed.

The finally block executes regardless of whatever happens.

Example: finally Block

```
try { var result = Sum(10, 20); // Sum is not defined yet } catch(ex) { document.getElementById("errorMessage").innerHTML = ex; } finally{ document.getElementById("message").innerHTML = "finally block executed"; }
```

throw

Use throw keyword to raise a custom error.

Example: throw Error

```
try { throw "Error occurred"; } catch(ex) { alert(ex); }
```

You can use JavaScript object for more information about an error.

Example: throw error with error info

```
try { throw { number: 101, message: "Error occurred" }; } catch (ex) { alert(ex.number + "- " + ex.message); }
```

JavaScript Strict Mode (With Examples)

JavaScript is a loosely typed (dynamic) scripting language. If you have worked with server side languages like Java or C#, you must be familiar with the strictness of the language. For example, you expect

the compiler to give an error if you have used a variable before defining it.

JavaScript allows strictness of code using "use strict" with ECMAScript 5 or later. Write "use strict" at the top of JavaScript code or in a function.

Example: strict mode

```
"use strict";
 var x = 1; // valid in strict mode
y = 1; // invalid in strict mode
```

The strict mode in JavaScript does not allow following things:

1. Use of undefined variables
2. Use of reserved keywords as variable or function name
3. Duplicate properties of an object
4. Duplicate parameters of function
5. Assign values to read-only properties
6. Modifying arguments object
7. Octal numeric literals
8. with statement
9. eval function to create a variable

Let look at an example of each of the above.

Use of undefined variables:

Example: strict mode

```
"use strict"; x = 1; // error
```

Use of reserved keyword as name:

Example: strict mode

```
"use strict"; var for = 1; // error var if = 1; // error
```

Duplicate property names of an object:

Example: strict mode

```
"use strict"; var myObj = { myProp: 100, myProp:"test strict mode" }; // error
```

Duplicate parameters:

Example: strict mode

```
"use strict"; function Sum(val, val){return val + val }; // error
```

Assign values to read-only property:

Example: strict mode

```
"use strict"; var arr = [1 ,2 ,3 ,4, 5]; arr.length = 10; // error
```

Modify arguments object:

Example: strict mode

```
"use strict"; function Sum(val1, val2){ arguments = 100; // error }
```

Octal literals:

Example: strict mode

```
"use strict"; var oct = 030; // error
```

with statement:

Example: strict mode

```
"use strict"; with (Math){ x = abs(200.234, 2); // error };
```

Eval function to create a variable:

Example: strict mode

```
"use strict"; eval("var x = 1");// error
```

Strict mode can be applied to function level in order to implement strictness only in that particular function.

Example: strict mode

```
x = 1; //valid function sum(val1, val2){ "use strict"; result = val1 + val2; //error return result; }
```

JavaScript Hoisting

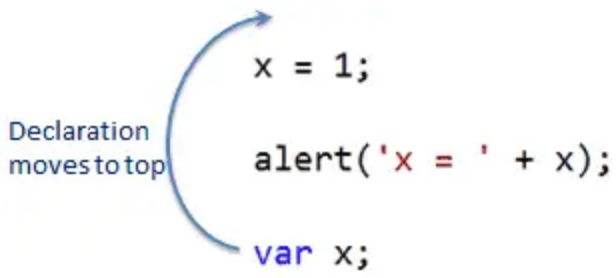
Hoisting is a concept in JavaScript, not a feature. In other scripting or server side languages, variables or functions must be declared before using it.

In JavaScript, variable and function names can be used before declaring it. The JavaScript compiler moves all the declarations of variables and functions at the top so that there will not be any error. This is called hoisting.

Example: Hoisting

```
x = 1; alert('x = ' + x); // display x = 1 var x;
```

The following figure illustrates hoisting.



JavaScript Hoisting

Also, a variable can be assigned to another variable as shown below.

Example: Hoisting

```
 x = 1;

y = x;
alert('x =' + x);alert('y =' + y);


var x; var y;
```

Hoisting is only possible with declaration but not the initialization. JavaScript will not move variables that are declared and initialized in a single line.

Example:

Hoisting not applicable for initialized variables
alert('x = ' + x);
// display
x = undefined
var x = 1;

As you can see in the above example, value of x will be undefined because var x = 1 is not hoisted.

Hoisting of Function

JavaScript compiler moves the function definition at the top in the same way as variable declaration.

Example: Function Hoisting

```
alert(Sum(5, 5)); // 10
function Sum(val1, val2)
{
return val1 + val2;
}
```

Please note that JavaScript compiler does not move function expression.

Example: Hoisting on function expression

```
Add(5, 5); // error
 var Add = function Sum(val1, val2)
 {
return val1 + val2;
}
```

Hoisting Functions Before Variables

JavaScript compiler moves a function's definition before variable declaration. The following example proves it.

Example: Function Hoisting Before Variables

```
alert(UseMe);
 var UseMe;
function UseMe()
{
alert("UseMe function called");
 }
```

As per above example, it will display UseMe function definition. So the function moves before variables.

Points to Remember :

JavaScript compiler moves variables and function declaration to the top and this is called hoisting.
Only variable declarations move to the top, not the initialization.
Functions definition moves first before variables.

Define Class in JavaScript

JavaScript ECMAScript 5, does not have class type. So it does not support full object oriented programming concept as other languages like Java or C#. However, you can create a function in such a way so that it will act as a class.

The following example demonstrates how a function can be used like a class in JavaScr3ipt.

Example: Class in JavaScript

```
function Person() { this.firstName = "unknown"; this.lastName = "unknown"; } var person1 = new Person(); person1.firstName = "Steve"; person1.lastName = "Jobs"; alert(person1.firstName + " " + person1.lastName); var person2 = new Person(); person2.firstName = "Bill"; person2.lastName = "Gates"; alert(person2.firstName + " " + person2.lastName );
```

In the above example, a `Person()` function includes firstName, lastName & age variables using this keyword. These variables will act like properties. As you know, we can create an object of any function using new keyword, so person1 object is created with new keyword. So now, Person will act as a class and person1 & person2 will be its objects (instances). Each object will hold their values separately because all the variables are defined with this keyword which binds

them to particular object when we create an object using new keyword.

So this is how a function can be used like a class in the JavaScript.

Add Methods in a Class

We can add a function expression as a member variable in a function in JavaScript. This function expression will act like a method of class.

Example: Method in Class

```
function Person() {
this.firstName = "unknown";
this.lastName = "unknown";
this.getFullName = function()
{ return this.firstName + " " + this.lastName; }
};
var person1 = new Person();
person1.firstName = "Steve";
person1.lastName = "Jobs";
alert(person1.getFullName());
var person2 = new Person();
person2.firstName = "Bill";
person2.lastName = "Gates";
alert(person2.getFullName());
```

In the above example, the Person function includes function expression that is assigned to a member variable getFullName. So now, getFullName() will act like a method of the Person class. It can be called using dot notation e.g. person1.getFullName().

Constructor

In the other programming languages like Java or C#, a class can have one or more constructors. In JavaScript, a function can have one or more parameters. So, a function with one or more parameters can be used like a constructor where you can pass parameter values at the time or creating an object with new keyword.

Example: Constructor

```
function Person(FirstName, LastName, Age)
{ this.firstName = FirstName || "unknown";
this.lastName = LastName || "unknown";
this.age = Age || 25;
this.getFullName = function ()
{ return this.firstName + " " + this.lastName; }
};
Varperson1=newPerson("James","Bond",50);
alert(person1.getFullName());
var person2 = new Person("Tom","Paul");
alert(person2.getFullName());
```

In the above example, the Person function includes three parameters FirstName, LastName and Age. These parameters are used to set the values of a respective property.

Please notice that parameter assigned to a property, if parameter value is not passed while creating an object using new then they will be undefined.

Properties with Getters and Setters

As you learned in the previous section, `Object.defineProperty()` method can be used to define a property with getter & setter.

The following example shows how to create a property with getter & setter.

Example: Property

```
function Person()
{ var _firstName = "unknown";
Object.defineProperties(this, { "FirstName": {
get: function () { return _firstName; },
set: function (value) { _firstName = value; }
}
});
 };
var person1 = new Person();
person1.FirstName = "Steve";
alert(person1.FirstName );
var person2 = new Person();
```

```
 person2.FirstName = "Bill";
alert(person2.FirstName );
```

In the above example, the Person() function creates a FirstName property by using Object.defineProperties() method. The first argument is this, which binds FirstName property to calling object. Second argument is an object that includes list of properties to be created. We have specified FirstName property with get & set function. You can then use this property using dot notation as shown above.

Read-only Property

Do not specify set function in order to create read-only property as shown below.

Example: Read-only Property

```
function Person(firstName) { var _firstName = firstName || "unknown"; Object.defineProperties(this, { "FirstName": { get: function () { return _firstName; } } }); }; var person1 = new Person("Steve"); //person1.FirstName = "Steve"; -- will not work
alert(person1.FirstName ); var person2 = new Person("Bill"); //person2.FirstName = "Bill"; -- will not work
alert(person2.FirstName );
```

Multiple Properties

Specify more than one property in defineProperties() method as shown below.

Example: Multiple Properties

```
function Person(firstName, lastName, age) { var _firstName = firstName || "unknown"; var _lastName = lastName || "unknown"; var _age = age || 25; Object.defineProperties(this, { "FirstName": { get: function () { return _firstName }, set: function (value) { _firstName = value } }, "LastName": { get: function () { return _lastName }, set: function (value) { _lastName = value } }, "Age": { get: function () { return _age }, set: function (value) { _age = value } } }); this.getFullName = function () { return this.FirstName + " " + this.LastName; }
```

```
}; var person1 = new Person(); person1.FirstName = "John";
person1.LastName = "Bond"; alert(person1.getFullName());
```

JavaScript Objects in Depth

You have already learned about JavaScript object in the JavaScript Basics section. Here, you will learn about object in detail.

As you know, object in JavaScript can be created using object literal, object constructor or constructor function. Object includes properties. Each property can either be assigned a literal value or a function.

Consider the following example of objects created using object literal and constructor function.

Example: JavaScript Object

```
// object literal var person = { firstName:'Steve',
lastName:'Jobs' }; // Constructor function function Student(){
this.name = "John"; this.gender = "Male"; this.sayHi =
function(){ alert('Hi'); } } var student1 = new Student();
console.log(student1.name); console.log(student1.gender);
student1.sayHi();
```

In the above example, `person` object is created using object literal that includes `firstName` and `lastName` properties and `student1` object is created using constructor function `Student` that includes name, gender and sayHi properties where function is assigned to `sayHi` property.

Any javascript function using which object is created is called constructor function.

Use Object.keys() method to retrieve all the properties name for the specified object as a string array.

Example: Edit Property Descriptor

```
function Student(){ this.title = "Mr."; this.name = "Steve";
this.gender = "Male"; this.sayHi = function(){ alert('Hi'); } }
var student1 = new Student(); Object.keys(student1);
```

Output:

```
["title", "name", "gender", "sayHi"]
```

Use for-in loop to retrieve all the properties of an object as shown below.

Example: Enumerable Properties

```
function Student(){ this.title = "Mr."; this.name = "Steve";
this.gender = "Male"; this.sayHi = function(){ alert('Hi'); } }
var student1 = new Student(); //enumerate properties of student1
for(var prop in student1){ console.log(prop); }
```

Output:

```
title name gender sayHi
```

Property Descriptor

In JavaScript, each property of an object has property descriptor which describes the nature of a property. Property descriptor for a particular

object's property can be retrieved using `Object.getOwnPropertyDescriptor()` method.

```
Object.getOwnPropertyDescriptor(object, 'property name')
```

The getOwnPropertyDescriptor method returns a property descriptor for a property that directly defined in the specified object but not inherited from object's prototytpe.

The following example display property descriptor to the console.

Example: Property Descriptor

```
var person = { firstName:'Steve', lastName:'Jobs' }; function
Student(){ this.name = "John"; this.gender = "Male"; this.sayHi
= function(){ alert('Hi'); } } var student1 = new Student();
console.log(Object.getOwnPropertyDescriptor(person,'firstName'))
; console.log(Object.getOwnPropertyDescriptor(student1,'name'));
console.log(Object.getOwnPropertyDescriptor(student1,'sayHi'));
```

Output:

```
Object {value: "Steve", writable: true, enumerable: true,
configurable: true} Object {value: "John", writable: true,
enumerable: true, configurable: true} Object {value: function,
writable: true, enumerable: true, configurable: true}
```

As you can see in the above output, the property descriptor includes the following 4 important attributes.

Attribute	Description

value	Contains an actual value of a property.
writable	Indicates that whether a property is writable or read-only. If true than value can be changed and if false then value cannot be changed and will throw an exception in strict mode
enumerable	Indicates whether a property would show up during the enumeration using for-in loop or Object.keys() method.
configurable	Indicates whether a property descriptor for the specified property can be changed or not. If true then any of this 4 attribute of a property can be changed using Object.defineProperty() method.

Object.defineProperty()

The Object.defineProperty() method defines a new property on the specified object or modifies an existing property or property descripto

```
Object.defineProperty(object, 'property name', descriptor)
```

The following example demonstrates modifying property descriptor.

Example: Edit Property Descriptor

```
'use strict' function Student(){ this.name = "Steve";
this.gender = "Male"; } var student1 = new Student();
Object.defineProperty(student1,'name', { writable:false} ); try
{ student1.name = "James"; console.log(student1.name); }
catch(ex) { console.log(ex.message); }
```

The above example, it modifies writable attribute of `name` property of `student1` object using `Object.defineProperty()`. So, `name` property can not be changed. If you try to change the value of `name` property then it would throw an exception in strict mode. In non-strict mode, it won't throw an exception but it also won't change a value of name property either.

The same way, you can change enumerable property descriptor as shown below.

Example: Edit Property Descriptor

```
function Student(){ this.name = "Steve"; this.gender = "Male"; }
var student1 = new Student(); //enumerate properties of student1
for(var prop in student1){ console.log(prop); } //edit
enumerable attributes of name property to false
Object.defineProperty(student1,'name',{ enumerable:false });
console.log('After setting enumerable to false:'); for(var prop
in student1){ console.log(prop); }
```

Output:

```
name gender After setting enumerable to false: gender
```

In the above example, it display all the properties using for-in loop. But, once you change the enumerable attribute to false then it won't display `name` property using for-in loop. As you can see in the output, after setting enumerable attributes of name property to false, it will not be enumerated using for-in loop or even Object.keys() method.

The Object.defineProperty() method can also be used to modify configurable attribute of a property which restrict changing any property descriptor attributes further.

The following example demonstrates changing configurable attribute.

Example: Edit Property Descriptor

```
'use strict'; function Student(){ this.name = "Steve";
this.gender = "Male"; } var student1 = new Student();
```

```
Object.defineProperty(student1,'name',{configurable:false});// set configurable to false try { Object.defineProperty(student1,'name',{writable:false}); // change writable attribute } catch(ex) { console.log(ex.message); }
```

In the above example, `Object.defineProperty(student1,'name',{configurable:false});` sets configurable attribute to false which make student1 object non configurable after that. `Object.defineProperty(student1,'name',{writable:false});` sets writable to false which will throw an exception in strict mode because we already set configurable to false.

Define New Property

The Object.defineProperty() method can also be used to define a new properties with getters and setters on an object as shown below.

Example: Define New Property

```
function Student(){ this.title = "Mr."; this.name = "Steve"; } var student1 = new Student(); Object.defineProperty(student1,'fullName',{ get:function(){ return this.title + ' ' + this.name; }, set:function(_fullName){ this.title = _fullName.split(' ')[0]; this.name = _fullName.split(' ')[1]; } }); student1.fullName = "Mr. John"; console.log(student1.title); console.log(student1.name);
```

Output:

```
Mr. John
```

JavaScript - this Keyword

The `this` keyword is one of the most widely used and yet confusing keyword in JavaScript. Here, you will learn everything about this keyword.

`this` points to a particular object. Now, which is that object is depends on how a function which includes 'this' keyword is being called.

Look at the following example and guess what the result would be?

```
<script> var myVar = 100; function WhoIsThis() { var myVar = 200; alert(myVar); // 200 alert(this.myVar); // 100 } WhoIsThis(); // inferred as window.WhoIsThis() var obj = new WhoIsThis(); alert(obj.myVar); </script>
```

The following four rules applies to `this` in order to know which object is referred by this keyword.

1. Global Scope
2. Object's Method
3. call() or apply() method
4. bind() method

Global Scope

If a function which includes 'this' keyword, is called from the global scope then `this` will point to the window object. Learn about global and local scope here.

Example: this keyword

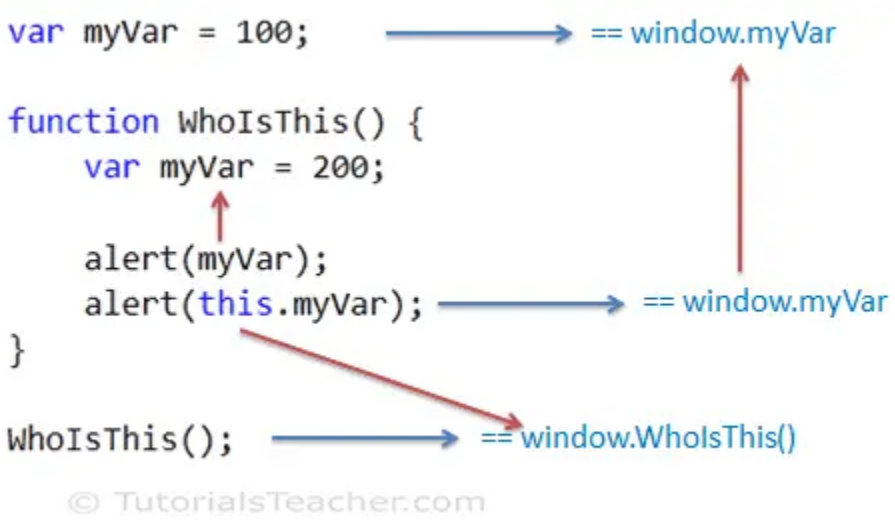
```
<script> var myVar = 100; function WhoIsThis() { var myVar = 200; alert("myVar = " + myVar); // 200 alert("this.myVar = " +
```

```
this.myVar);    //    100    }    WhoIsThis();    //    inferred    as
window.WhoIsThis() </script>
```

Try it

In the above example, a function `WhoIsThis()` is being called from the global scope. The global scope means in the context of window object. We can optionally call it like `window.WhoIsThis()`. So in the above example, `this` keyword in `WhoIsThis()` function will refer to window object. So, `this.myVar` will return 100. However, if you access `myVar` without `this` then it will refer to local `myVar` variable defined in `WhoIsThis()` function.

The following figure illustrates the above example.



this in JavaScript

In the strict mode, value of 'this' will be undefined in the global scope.

'this' points to global window object even if it is used in an inner function. Consider the following example.

Example: this keyword inside inner function

```
var myVar = 100; function SomeFunction() { function WhoIsThis() { var myVar = 200; alert("myVar = " + myVar); // 200 alert("this.myVar = " + this.myVar); // 100 } WhoIsThis(); } SomeFunction();
```

So, if 'this' is used inside any global function and called without dot notation or using `window.` then `this` will refer to global object which is default window object.

this Inside Object's Method

As you have learned here, you can create an object of a function using new keyword. So, when you create an object of a function using new keyword then `this` will point to that particular object. Consider the following example.

Example: this keyword

```
var myVar = 100; function WhoIsThis() { this.myVar = 200; } var obj1 = new WhoIsThis(); var obj2 = new WhoIsThis(); obj2.myVar = 300; alert(obj1.myVar); // 200 alert(obj2.myVar); // 300
```

In the above example, `this` points to `obj1` for `obj1` instance and points to `obj2` for `obj2` instance. In JavaScript, properties can be attached to an object dynamically using dot notation. Thus, `myVar` will be a property of both the instances and each will have a separate copy of `myVar`.

Now look at the following example.

Example: this keyword

```
var myVar = 100; function WhoIsThis() { this.myVar = 200; this.display = function(){ var myVar = 300; alert("myVar = " +
```

```
myVar); // 300 alert("this.myVar = " + this.myVar); // 200 }; }
var obj = new WhoIsThis(); obj.display();
```

In the above example, `obj` will have two properties `myVar` and `display`, where display is a function expression. So, `this` inside display() method points to `obj` when calling `obj.display()`.

`this` behaves the same way when object created using object literal, as shown below.

Example: this keyword

```
var myVar = 100; var obj = { myVar : 300,              whoIsThis:
function(){   var   myVar   =   200;   alert(myVar);   //   200
alert(this.myVar); // 300      } }; obj.whoIsThis();
```

call() and apply()

In JavaScript, a function can be invoked using () operator as well as call() and apply() method as shown below.

Example: Function call

```
function   WhoIsThis()   {   alert('Hi');   }   WhoIsThis();
WhoIsThis.call(); WhoIsThis.apply();
```

In the above example, WhoIsThis(), WhoIsThis.call() and WhoIsThis.apply() executes a function in the same way.

The main purpose of call() and apply() is to set the context of `this` inside a function irrespective whether that function is being called in the global scope or as object's method.

You can pass an object as a first parameter in call() and apply() to which the `this` inside a calling function should point to.

The following example demonstrates the call() & apply().

Example: call() & apply()

```
var myVar = 100; function WhoIsThis() { alert(this.myVar); } var obj1 = { myVar : 200 , whoIsThis: WhoIsThis }; var obj2 = { myVar : 300 , whoIsThis: WhoIsThis }; WhoIsThis(); // 'this' will point to window object WhoIsThis.call(obj1); // 'this' will point to obj1 WhoIsThis.apply(obj2); // 'this' will point to obj2 obj1.whoIsThis.call(window); // 'this' will point to window object WhoIsThis.apply(obj2); // 'this' will point to obj2
```

As you can see in the above example, when the function WhoIsThis is called using () operator (like WhoIsThis()) then `this` inside a function follows the rule- refers to window object. However, when the WhoIsThis is called using call() and apply() method then `this` refers to an object which is passed as a first parameter irrespective of how the function is being called.

Therefore, `this` will point to obj1 when a function got called as `WhoIsThis.call(obj1)`. In the same way, `this` will point to obj2 when a function got called like `WhoIsThis.apply(obj2)`

bind()

The bind() method was introduced since ECMAScript 5. It can be used to set the context of 'this' to a specified object when a function is invoked.

The bind() method is usually helpful in setting up the context of this for a callback function. Consider the following example.

Example: bind()

```
var myVar = 100; function SomeFunction(callback) { var myVar = 200; callback(); }; var obj = { myVar: 300, WhoIsThis : function() { alert("'this' points to " + this + ", myVar = " +
```

```
this.myVar);        }        };        SomeFunction(obj.WhoIsThis);
SomeFunction(obj.WhoIsThis.bind(obj));
```

In the above example, when you pass `obj.WhoIsThis` as a parameter to the SomeFunction() then `this` points to global window object insted of obj, because `obj.WhoIsThis()` will be executed as a global function by JavaScript engine. You can solve this problem by explicitly setting `this` value using bind() method. Thus, `SomeFunction(obj.WhoIsThis.bind(obj))` will set `this` to `obj` by specifying `obj.WhoIsThis.bind(obj)`.

Precedence

So these 4 rules applies to `this` keyword in order to determine which object `this` refers to. The following is precedence of order.

1. bind()
2. call() and apply()
3. Object method
4. Global scope

So, first check whether a function is being called as callback function using bind()? If not then check whether a function is being called using call() or apply() with parameter? If not then check whether a function is being called as an object function? Otherwise check whether a function is being called in the global scope without dot notation or using window object.

Thus, use these simple rules in order to know which object the 'this' refers to inside any function.

DOM (Document Object Model)

JavaScript Document Object

JavaScript Document object is an object that provides access to all HTML elements of a document. When an HTML document is loaded into a browser window, then it becomes a document object.

The document object stores the elements of an HTML document, such as HTML, HEAD, BODY, and other HTML tags as objects.

A document object is a child object of the Window object, which refers to the browser.

You can access a document object either using `window.document` property or using object directly.

See, What we can do with Document Object:

- Suppose you have created a FORM in an HTML document using the FORM element and added some text fields and Buttons on the Form. On Submitting the Form you want to validate the input or display input on another page. In this situation, you can use a document object which is a child object of the window object. Using the document object, you can access the elements of the form and validate the Input.
- The Document Object provides different collection elements, such as anchor and Links which helps you to count the number of specific elements on a form.
- The Document Object also provides various properties such as `URL`, `bgcolor`, etc. which will allow you to retrieve the details of a document and various methods such as `open()`, `close()`, `write()`, g`etElementById()`, `getElementByName()`, etc. which allow you to perform various tasks like opening an HTML document to display output and writing a text on Web Page.
- The Document Object also allows you to create cookies for a webpage by providing a property named cookie. A cookie stores information about the user's computer, which helps in accessing visited websites.

Now let's explore document object methods and properties.

JavaScript Document Object Properties

As we know, a property of an object is the value associated with the object. The property is accessed by using the following notation:

where objectName is the name of the object and propertyName is the name of its property.

Now we will show you the properties of document object in the table given below:

Properties	Description
cookie	returns a report that contains all the visible and unexpired cookies associated with the document
domain	returns the domain name of the server from which the document has originated
lastModified	returns the date on which document was last modified
documentMode	returns the mode used by the browser to process the document
readyState	returns the loading status of the document.
referrer	returns the URL of the documents referred to in an HTML document

title	returns the name of the HTML document defined between the starting and ending tags of the TITLE element
URL	returns the full URL of the HTML document.

Let's take an example, to see the document object properties in action:

JavaScript Document Object Methods

JavaScript Document object also provides various methods to access HTML elements.

Now we will show you some of the commonly used methods of the document object:

Methods	Description	Syntax
`open()`	opens an HTML document to display the output	`document.open(mimetype, replace)` Copy
`close()`	closes an HTML document	`document.close()` Copy

`write()`	Writes HTML expressions or JavaScript code into an HTML document	`document.write(exp1, exp2, ...)` Copy
`writeln()`	write a new line character after each HTML expressions or JavaScript Code	`document.writeln(exp1, exp2, ...)` Copy
`getElementById()`	returns the reference of first element of an HTML document with the specified id.	`document.getElementById(id)` Copy
`getElementByName()`	returns the reference of an element with the specified name	`document.getElementsByName(name)` Copy

`getElementsByTagName()`	returns all elements with the specified tagname	document.getElementsByTagName(tagname) Copy

BOM (Browser Object Model)

The **Browser Object Model** (BOM) is used to interact with the browser.

The default object of browser is window means you can call all the functions of window by specifying window or directly.

JavaScript Window Object

JavaScript Window is a global Interface (object type) which is used to control the browser window lifecycle and perform various operations on it.

A global variable window, which represents the current browser window in which the code is running, is available in our JavaScript code and can be directly accessed by using `window` literal.

It represents a window containing a webpage which in turn is represented by the document object. A window can be a new window, a new tab, a frame set or individual frame created with JavaScript.

In case of multi tab browser, a window object represents a single tab, but some of its properties like `innerHeight`, `innerWidth` and methods like `resizeTo()` will affect the whole browser window.

Whenever you open a new window or tab, a window object representing that window/tab is automatically created.

The properties of a window object are used to retrieve the information about the window that is currently open, whereas its methods are used to perform specific tasks like opening, maximizing, minimizing the window, etc.

To understand the window object, let's use it to perform some operations and see how it work.

Using JavaScript Window Object

Let's use the JavaScript window object to create a new window, using the `open()` method. This method creates a new window and returns an object which further can be used to manage that window.

```
<!doctype html>
	<head>
		<title>Open a new Window</title>
	</head>
	<body>
		<h3>Click to open a new Window:</h3>

	<button onclick="createWindow()">Open a Window</button>
	<p id="result"></p>

	<script>
	function createWindow() {
	   let url = "https://studytonight.com";
	   let win = window.open(url, "My New Window", "width=300,
height=200");
		document.getElementById("result").innerHTML = win.name +
" - " + win.opener.location;
	}
	</script>

	</body>
```

</html>

In the code above, we have used the window object of the existing window to create a new window using the `open()` method. In the `open()` method we can provide the URL to be opened in the new window(we can keep it blank as well), name of the window, the width and the height of the window to be created.

Following is the syntax for the window object open() method:

We can provide, as many properties as we want, while creating a new window.

When the `open()` method is executed, it returns the reference of the window object for the new window created, which you can assign to a variable, like we have done in the code above. We have assigned the value returned by the `window.open()` method to the `win` variable.

The we have used the `win` variable to access the new window, like getting the name of the window, getting location of the window which opened the new window etc.

There are many properties and methods for the window object, which we have listed down below.

Find Dimensions of a Window

We can get the height and width of a window by using the window object built-in properties.

We can also access the document object using a window object(`window.document`) which gives us access to the HTML document, hence we can add new HTML element, or write any content to the document, like we did in the example above.

JavaScript Window Object Properties

The window object properties refer to the variables created inside the windows object.

In JavaScript, all the available data is attached to the window object as properties.

We can access window object's properties like: `window.propertyname` where propertyname is the name of property.

A table of most popular window object properties is given below:

```
<!doctype html>
        <head>
                <title>Open a new Window</title>
        </head>
        <body>
                <h3>Click to open a new Window:</h3>

    <button onclick="createWindow()">Open a Window</button>
    <p id="result"></p>

    <script>
    function createWindow() {
       let url = "https://studytonight.com";
       let win = window.open(url, "My New Window", "width=300, height=200");
       document.getElementById("result").innerHTML = win.name + " - " +
win.opener.location;
    }
    </script>

        </body>
</html>
```

Property	Description
closed	returns a boolean value that specifies whether a window has been closed or not
document	specifies a document object in the window.
history	specifies a history object for the window.
frames	specifies an array of all the frames in the current window
defaultStatus	specifies the default message that has to be appeared in the status bar of Window.
innerHeight	specifies the inner height of the window's content area.
innerWidth	specifies the inner width of the window's content area.
length	specifies the number of frames contained in the window.

location	specifies the location object for window
name	specifies the name for the window
top	specifies the reference of the topmost browser window.
self	returns the reference of current active frame or window.
parent	returns the parent frame or window of current window.
status	specifies the message that is displayed in the status bar of the window when an activity is performed on the Window.
screenleft	specifies the x coordinate of window relative to a user's monitor screen
screenTop	Specifies the y coordinate of window relative to a user's monitor screen
screenX	specifies the x coordinate for window relative to a user's monitor screen

screenY	Specifies the y coordinate for window relative to a user's monitor screen

Let's take an example to see a few of these properties in action:

```
<!doctype html>
        <head>
                <title>Add content to Window</title>
        </head>
        <body>
      <script>
        let h = window.outerHeight
        let w = window.outerWidth
        window.document.write("Height and width of the window are: "+h+" and "+w)
      </script>

        </body>
</html>
```

In the example above, we have created a new window, just to make you familiar with new window creation and also, to make you understand that when we create a new window, then the current window has a different window object and the new window will have a separate window object.

We have then tried to access some properties of the new window created.

TASK: Try some more windows property in the above code playground to practice and understant how window properties work.

JavaScript Window Object Methods

The window object methods refer to the functions created inside the Window Object, which can be used to perform various actions on the browser window, such as how it displays a message or gets input from the user.

Below we have listed down some of the most commonly used window object methods:

Method	Description
`alert()`	specifies a method that displays an alert box with a message an OK button.
`blur()`	specifies a method that removes the focus from the current window.
`clearInterval()`	specifies a method that clears the timer, which is set by using setInterval() method.
`close()`	specifies a method that closes the current window.
`confirm()`	specifies a method that displays a dialogue box with a message and two buttons OK and Cancel

`focus()`	specifies a method that sets the focus on current window.
`open()`	specifies a method that opens a new browser window
`moveTo()`	specifies a method that moves a window to a specified position
`moveBy()`	specifies a Method that moves a window relative to its current position.
`prompt()`	specifies a method that prompts for input.
`print()`	specifies a method that sends a print command to print the content of the current window.
`setTimeout()`	specifies a method that evaluates an expression after a specified number of milliseconds.
`setInterval()`	specifies a method that evaluates an expression at specified time intervals (in milliseconds)

`resizeBy()`	specifies the amount by which the window will be resized
`resizeTo()`	used for dynamically resizing the window
`scroll()`	scrolls the window to a particular place in document
`scrollBy()`	scrolls the window by the given value
`stop()`	this method stops window loading

Let's take an example and see a few of the above methods in action:

```
<!doctype html>
	<head>
		<title>JS Window Method Examples</title>
	</head>
	<body>
		<h3>Perform various operations on Window object</h3>

	<button onclick="createWindow()">Open a Window</button>
	<button onclick="closewin()">Close the Window</button>
	<button onclick="showAlert()">Show Alert in Window</button>

	<script>
	var win;
```

```
        // Open window
        function createWindow() {
          win = window.open("", "My Window", "width=500,
height=200");
        }

        function showAlert() {
            if(win == undefined) {
                // show alert in main window
                window.alert("First create the new window, then show alert
in it");
            }
            else {
                win.alert("This is an alert");
            }
        }

        // Close window
        function closewin(){
            win.close();
        }
        </script>

        </body>
</html>
```